Chapter 6

Machine Learning Fundamentals

6.1 Introduction

Machine learning has become a central part of the quantitative toolkit in finance. It provides methods for discovering patterns in data, forecasting outcomes, reducing dimensionality, and classifying observations. In principle its function is simple: use historical data to learn relationships that generalize to new situations. In practice this is hard, especially in finance, where data are noisy, relationships shift over time, and the cost of mistakes can be large.

This chapter introduces the main ideas of machine learning for readers who are mathematically trained but may be new to the subject, focusing on concepts and formulas rather than library calls. Two small worked examples run through the chapter: a regression problem relating a firm's leverage to its credit spread, and a classification problem predicting loan default. The discussion covers supervised and unsupervised learning, prediction and classification, overfitting, regularization, evaluation strategies, and the bias-variance trade-off.

By the end of the chapter, the reader will have seen how a model's parameters are found (by a closed-form solution or gradient descent), how distance-based, margin-based, probabilistic, and tree-based algorithms attack the same prediction problem from different angles, and how to choose among them and their hyperparameters without overfitting the selection itself. Every applied chapter that follows builds on this vocabulary: the credit models of Chapter 9, the portfolio methods of Chapter 12, the trading strategies of Chapters 13 and 14, the fraud detection systems of Chapter 15, the text-based methods of Chapter 16, the deep-learning extensions of Chapter 17, the explainability tools of Chapter 18, and the end-to-end case study of Chapter 21.

6.2 The Learning Problem

Two questions come before any algorithm: what a learning system does, and what kind of problem is being posed to it.

6.2.1 What Machine Learning Is

Suppose an analyst has a decade of loan-application data and wants a system that predicts default risk better than hand-written underwriting rules, and that improves as data arrives instead of being rewritten whenever economic conditions shift. Machine learning is the branch of statistics and computer science built for this: models that improve with experience, rather than a fixed decision procedure encoded by hand. A model is given data, it learns parameters from those data by minimizing some measure of error, and it is then used to make predictions or decisions on new data. The model may be used for regression, classification, clustering, dimensionality reduction, or anomaly detection.

This setup is not new. Least-squares regression, dating to Gauss and Legendre in the early 1800s, is a machine learning algorithm by this definition: it minimizes squared error over a set of parameters using data. What distinguishes the modern field is less the principle, minimizing a loss function over parameters, than the scale of data and computation now available. That scale favors flexible, high-capacity models (trees,

Wulin.Suo@Queensu.ca

ensembles, neural networks) with thousands or millions of parameters, which are harder to interpret than the small, transparent linear models of classical statistics.

In finance, machine learning can be used for several purposes. It can forecast returns or volatility, classify whether a firm is likely to default, detect fraudulent transactions, or extract sentiment from news articles. In each case the model must learn from observed patterns and generalize to future cases, a harder task than describing the past.

The challenge is that finance is not a static environment. Relationships can change because of policy shifts, market regimes, investor behavior, or macroeconomic conditions. A model that performs well in one regime may fail in another. Machine learning in finance must therefore be tested under realistic conditions, a theme that runs through this chapter and every applied chapter after it.

6.2.2 Supervised and Unsupervised Learning

Machine learning problems are often divided into supervised and unsupervised learning. In supervised learning, the outcome of interest, called the label or target, is observed in the training data. For example, one may have historical data on firm characteristics and a label indicating whether default occurred. The task is to learn a function $\hat{f}$ mapping features x to a target y, so that $\hat{f}(x) \approx y$ on new, unseen data. Common examples include linear regression, logistic regression, decision trees, and neural networks.

In unsupervised learning, no explicit target is available. The goal is to discover structure in the data. Clustering methods group similar observations together, while dimensionality reduction methods compress the data into lower-dimensional representations. In finance, unsupervised learning can help identify groups of similar assets, detect unusual trading patterns, or reduce the dimensionality of large datasets.

A third category, semi-supervised learning, uses a small amount of labeled data with a much larger amount of unlabeled data. This is common in finance, where a reliable label (confirming that a transaction was fraudulent, say, which may take a lengthy investigation) is expensive relative to the raw data. A related distinction is between discriminative models, which learn the boundary or function that predicts labels directly (linear and logistic regression and most models in this chapter), and generative models, which learn the full probability distribution of the data and can generate synthetic examples resembling the training data. Naive Bayes, introduced later in this chapter, is a simple generative model; large language models, discussed in Chapter 16, are far more elaborate examples of the same generative principle applied to text.

The choice of method depends on the question. Predicting default calls for supervised learning; finding hidden structure in a large set of securities may call for unsupervised learning. The chapter proceeds in roughly that order: supervised regression and classification first, since the applied chapters use them most, then unsupervised clustering and dimensionality reduction, and finally neural networks, which can serve either purpose depending on how they are trained.

6.3 Core Supervised Models

This part builds the standard supervised toolkit, each model worked numerically before the next is introduced.

6.3.1 Worked Example 6.1: Regression

Suppose a credit analyst wants to relate a firm's leverage, measured as debt-to-EBITDA, to the credit spread the market demands on its bonds, measured in basis points. Table 6.1 shows five observations.

Debt/EBITDA (x)	2	3	4	5	6
Credit spread, bps (y)	120	150	210	260	300

Table 6.1: Leverage and credit spread for five hypothetical issuers

Linear regression models the target as a linear function of the feature plus noise, $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. The ordinary least squares (OLS) estimator chooses β_0, β_1 to minimize the sum of squared residuals,

$$(\hat{\beta}_0, \hat{\beta}_1) = \arg\min_{\beta_0,\beta_1} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2 , \tag{6.1}$$

Wulin.Suo@Queensu.ca

which has the closed-form solution

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}. \tag{6.2}$$

With $\bar{x} = 4$ and $\bar{y} = 208$, the data in Table 6.1 give $\sum_i (x_i - \bar{x})(y_i - \bar{y}) = 470$ and $\sum_i (x_i - \bar{x})^2 = 10$, so

$$\hat{\beta}_1 = \frac{470}{10} = 47, \qquad \hat{\beta}_0 = 208 - 47(4) = 20.$$

The fitted model is $\widehat{\text{spread}} = 20 + 47 \times \text{leverage}$: each additional turn of debt-to-EBITDA is associated with roughly 47 additional basis points of credit spread. The fitted values are 114, 161, 208, 255, and 302 bps, close to the observed 120, 150, 210, 260, and 300. Figure 6.1 plots the five issuers with the fitted line, showing the tightness of the fit and each residual.

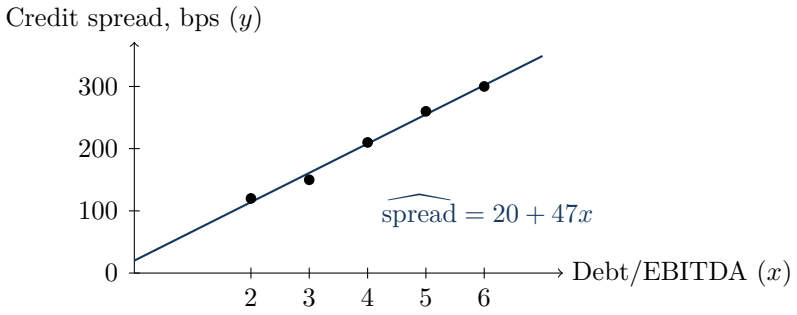


Figure 6.1: Debt-to-EBITDA leverage and credit spread for the five hypothetical issuers in Table 6.1 (black dots), with the OLS-fitted line $\widehat{\text{spread}} = 20 + 47x$ overlaid as a solid line

The residual sum of squares is

$$\text{SSE} = \sum_i (y_i - \hat{y}_i)^2 = 190,$$

and the total sum of squares around the mean is

$$\text{SST} = \sum_i (y_i - \bar{y})^2 = 22{,}280,$$

giving

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{190}{22{,}280} \approx 0.991,$$

so leverage explains about 99% of the variation in credit spread in this small, deliberately clean example. In sklearn, the same fit is obtained with

```
from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([120, 150, 210, 260, 300])
model = LinearRegression().fit(X, y)
print(model.intercept_, model.coef_)   # 20.0 [47.]
print(model.score(X, y))               # 0.9915...
```

Real credit spread data would show far more scatter; the toy example makes the mechanics of fitting and evaluating a linear model transparent before real-world noise is added.

Wulin.Suo@Queensu.ca

6.3.2 Standard Errors, Confidence Intervals, and p-Values for Coefficients

The point estimate $\hat{\beta}_1 = 47$ says nothing about how much confidence it deserves, and a workflow that reports only a point estimate and an R^2 leaves useful information on the table. With $n = 5$ observations and two estimated parameters $(\hat{\beta}_0, \hat{\beta}_1)$, there are $n - 2 = 3$ residual degrees of freedom, and the estimated noise variance is

$$\hat{\sigma}^2 = \text{SSE}/(n-2) = 190/3 \approx 63.33.$$

The standard error of the slope, which measures how much $\hat{\beta}_1$ would be expected to vary across hypothetical repeated samples of five issuers drawn from the same underlying relationship, is

$$SE(\hat{\beta}_1) = \sqrt{\frac{\hat{\sigma}^2}{\sum_i (x_i - \bar{x})^2}} = \sqrt{\frac{63.33}{10}} \approx 2.52.$$

Dividing the estimate by its standard error gives the t-statistic, $t = 47/2.52 \approx 18.68$, which under the null hypothesis $H_0 : \beta_1 = 0$ (leverage carries no information about credit spread) follows a t-distribution with 3 degrees of freedom. The associated p-value, the probability of a t-statistic this extreme by chance if H_0 were true, is $p \approx 0.0003$, far below the conventional 5% threshold: strong evidence, within this small and clean sample, that the slope is nonzero. A 95% confidence interval for the slope is

$$\hat{\beta}_1 \pm t^*_{0.975,3} \cdot SE(\hat{\beta}_1) = 47 \pm (3.182)(2.52) \approx (39.0,\ 55.0),$$

using the t-distribution critical value $t^*_{0.975,3} \approx 3.182$. The correct reading is that if this sampling and estimation procedure were repeated many times, about 95% of the resulting intervals would contain the true relationship between leverage and spread, not that there is a 95% chance the true β_1 falls in this specific interval. Applying the same steps to the intercept gives $SE(\hat{\beta}_0) \approx 10.68$, $t \approx 1.87$, and $p \approx 0.158$: the intercept is not statistically distinguishable from zero at the 5% level. That is unsurprising: only one of the five leverage values ($x = 2$) is anywhere near $x = 0$, so the data say little about the spread a zero-leverage issuer would command.

The contrast between slope and intercept generalizes: significance is coefficient-specific, and a model can pin down some relationships far more precisely than others. The overfitting concern of this chapter applies here too. Fitting a regression with many candidate features and reporting only those with p-values below 0.05 overstates the evidence, since testing enough coefficients against noise eventually produces a spuriously "significant" one, the multiple-testing concern revisited in Chapter 7's discussion of pairs selection for cointegration trading. A single small p-value is far more persuasive when it was the only hypothesis tested than when it is the best of dozens.

6.3.3 How Models Learn: Gradient Descent

The closed-form OLS solution above exists because linear regression's loss function has a special structure: it is quadratic in the parameters, so its minimum can be found in one step by solving a system of linear equations. Most machine learning models, including logistic regression and neural networks, lack this property, and their parameters must be found iteratively with an optimization algorithm called gradient descent. Figure 6.2 shows the idea for a single generic parameter, written θ there and standing for whichever parameters a given model has to fit.

Starting from an initial guess, the algorithm repeatedly takes a small step in the direction that most reduces the loss, the negative of the gradient (the vector of partial derivatives) of the loss with respect to each parameter, and stop once further steps no longer improve the loss. For the mean squared error loss

$$\mathcal{L}(\beta_0, \beta_1) = \frac{1}{n} \sum_i (\beta_0 + \beta_1 x_i - y_i)^2,$$

the gradients are

$$\frac{\partial \mathcal{L}}{\partial \beta_0} = \frac{2}{n} \sum_i (\hat{y}_i - y_i), \qquad \frac{\partial \mathcal{L}}{\partial \beta_1} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)\, x_i,$$

Wulin.Suo@Queensu.ca

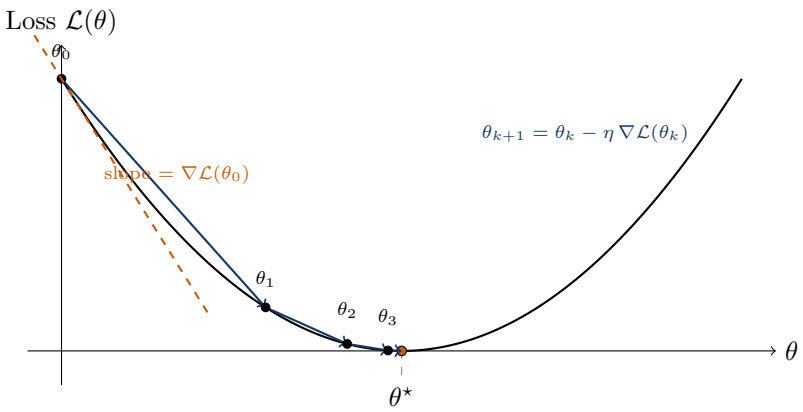


Figure 6.2: The idea of gradient descent. Starting from an initial guess θ_0, each step moves against the local slope of the loss curve, the negative gradient, illustrated here by the tangent line at θ_0, producing a sequence $\theta_0, \theta_1, \theta_2, \ldots$ that descends toward the minimizer $\theta^\star$. Steps are large where the curve is steep and shrink automatically as the curve flattens near the minimum, the pattern seen numerically in Table 6.2 below.

and each iteration updates the parameters by

$$\beta_0 \leftarrow \beta_0 - \eta\frac{\partial\mathcal{L}}{\partial\beta_0}, \qquad \beta_1 \leftarrow \beta_1 - \eta\frac{\partial\mathcal{L}}{\partial\beta_1}, \tag{6.3}$$

where η is the learning rate, a small positive number controlling the step size. Applying this to the same credit-spread data as Section 6.3.1, starting from $\beta_0 = \beta_1 = 0$ with a learning rate of $\eta = 0.01$, produces the trajectory in Table 6.2.

Iteration	β_0	β_1
1	4.16	18.52
2	6.76	30.04
3	8.38	37.21
10	11.05	48.55
1,000	18.91	47.24
10,000	20.00	47.00

Table 6.2: Gradient descent converging toward the closed-form OLS solution

After 10,000 iterations, gradient descent has converged to $\beta_0 \approx 20.00$ and $\beta_1 \approx 47.00$, matching the closed-form OLS solution of Section 6.3.1 up to rounding. For a quadratic loss like this one, gradient descent is guaranteed to converge to the same global minimum, provided the learning rate is small enough to avoid overshooting. The learning rate itself involves a trade-off that recurs throughout machine learning: too small, and convergence (as seen here) can require many thousands of iterations; too large, and the updates overshoot the minimum and diverge. In practice η is one of the most important hyperparameters to tune, including for the neural networks later in this chapter, and it is rarely fixed throughout training; adaptive variants that shrink the rate over time, or use different rates for different parameters, are standard in deep learning software. Gradient descent is worth understanding even where a closed form exists because it scales to problems with none: logistic regression's log-loss, a neural network's loss with millions of parameters, and many other models in this book are fit with it or a close variant, arguably making it the most important algorithm in modern machine learning.

Wulin.Suo@Queensu.ca

6.3.4 Worked Example 6.2: Classification

Regression predicts a continuous target; classification predicts a discrete one, such as whether a borrower defaults. Logistic regression models the probability of the positive class as

$$p(x) = \Pr(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}, \tag{6.4}$$

so that $p(x)$ is always between 0 and 1, unlike a linear model's unbounded output. The parameters are typically chosen to minimize the log-loss (negative log-likelihood),

$$\mathcal{L}(\beta) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log p(x_i) + (1 - y_i) \log \big(1 - p(x_i)\big) \Big]. \tag{6.5}$$

A note on terminology the reader will meet in industry: linear and logistic regression both belong to the generalized linear model (GLM) family, which pairs a linear combination of features with a link function and an outcome distribution suited to the target. Ordinary regression uses an identity link with a normal outcome, logistic regression a log-odds link with a binary outcome, and other widely used members a log link with Poisson-distributed counts or Gamma-distributed positive amounts. GLMs remain the regulatory workhorse of insurance pricing because their fitted coefficients read as multiplicative rating factors.

Suppose a lender fits this model to 10 past loan applicants, using a single feature, the applicant's debt-to-income ratio, and classifies an applicant as a predicted default (1) when the fitted probability $\hat{p}$ is at least 0.5 and as a predicted no default (0) otherwise; the fit itself is worked through at the end of this section. Table 6.3 shows each applicant's ratio, fitted probability, and actual and predicted outcomes.

Applicant	1	2	3	4	5	6	7	8	9	10
Debt/income (%)	48	26	44	40	33	37	22	18	31	15
Fitted $\hat{p}$	0.932	0.127	0.857	0.724	0.382	0.585	0.060	0.027	0.291	0.015
Actual default	1	0	1	0	0	1	0	0	1	0
Predicted default	1	0	1	1	0	1	0	0	0	0

Table 6.3: Debt-to-income ratio, fitted default probability, and actual versus predicted default for 10 loan applicants

Comparing the two rows produces a confusion matrix: 3 true positives (TP; applicants 1, 3, 6), 1 false negative (FN; applicant 9, an actual default the model missed), 1 false positive (FP; applicant 4, a good loan flagged as risky), and 5 true negatives (TN). Table 6.4 arranges these four counts into the standard 2×2 layout, with the model's predictions as rows and the actual outcomes as columns.

	Actual	
Predicted	Default (1)	No Default (0)
Default (1)	$TP = 3$	$FP = 1$
No Default (0)	$FN = 1$	$TN = 5$

Table 6.4: Confusion matrix for the 10 loan applicants in Table 6.3

Wulin.Suo@Queensu.ca

From these four counts, the standard classification metrics follow directly:

$$\begin{aligned}
\text{Accuracy} &= \frac{TP + TN}{n} = \frac{3 + 5}{10} = 0.80, \\
\text{Precision} &= \frac{TP}{TP + FP} = \frac{3}{4} = 0.75, \\
\text{Recall (TPR)} &= \frac{TP}{TP + FN} = \frac{3}{4} = 0.75, \\
\text{F1} &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 0.75, \\
\text{FPR} &= \frac{FP}{FP + TN} = \frac{1}{6} \approx 0.167.
\end{aligned}$$

Each metric answers a different question. Accuracy answers "what fraction of predictions were correct," which misleads when defaults are rare, since always predicting "no default" scores highly while being useless. Precision answers "of the loans flagged as risky, how many actually defaulted," which matters when false positives (turning away good customers) are costly. Recall answers "of the loans that actually defaulted, how many did the model catch," which matters when false negatives (missed defaults) are costly. A receiver operating characteristic (ROC) curve plots the true positive rate against the false positive rate as the classification threshold varies, and the area under this curve (AUC) summarizes performance across all thresholds rather than the single one used in Table 6.3.

Worked Example 6.3: Fitting the Logistic Regression and Computing the AUC

The probabilities in Table 6.3 come from choosing β_0 and β_1 to maximize the likelihood of the observed defaults under Equation (6.4), which is the same as minimizing the log-loss (6.5). There is no closed form, so the minimum is found numerically, by gradient descent or, as statistical software usually does, by Newton's method. At the minimum two first-order conditions hold, $\sum_i (y_i - \hat{p}_i) = 0$ and $\sum_i x_i (y_i - \hat{p}_i) = 0$: the fitted probabilities add up to the observed number of defaults, four, and the residuals are uncorrelated with the feature. For the ten applicants the solution is

$$\log \frac{\hat{p}}{1 - \hat{p}} = -7.2885 + 0.2063 \times \text{DTI},$$

with DTI, the debt-to-income ratio, in percentage points. The slope is read on the log-odds scale: each additional point of debt-to-income multiplies the odds of default by $e^{0.2063} = 1.229$, an increase of about 23%. The fitted probability crosses 0.5 where the log-odds are zero, at DTI $= 7.2885/0.2063 = 35.33\%$, which is why applicants 1, 3, 4, and 6 (ratios of 37% and above) are predicted to default and applicant 9, a defaulter at 31%, is missed. For applicant 6, the log-odds are $-7.2885 + 0.2063 \times 37 = 0.345$ and $\hat{p} = 1/(1 + e^{-0.345}) = 0.585$. The log-loss at the optimum is 0.401, against 0.673 for a model that ignores the feature and gives every applicant the base rate of 40%.

Two caveats belong with these numbers. First, ten observations pin the slope down poorly: its standard error, from the inverse of the Fisher information matrix, is 0.126, so $z = 1.64$ and $p = 0.10$, and the evidence that the ratio matters is suggestive rather than conclusive. Second, the fit is unpenalized, which works only because the data are not perfectly separable: applicant 4, who repaid, has a higher ratio than defaulters 6 and 9. Had every defaulter had a higher ratio than every non-defaulter, the likelihood would keep rising as $\beta_1 \to \infty$, no maximum would exist, and any reported coefficients would be artifacts of the optimizer's stopping rule or of a penalty. Library defaults matter here, as Section 6.4.1 shows for the lasso: scikit-learn's `LogisticRegression` adds a ridge penalty unless told otherwise (`C=1` by default), which shrinks this fit slightly to $-7.1781 + 0.2031 \times \text{DTI}$, and the unpenalized fit requires `C=np.inf`. This model is the workhorse of Chapters 9, 15, 18, and 21: credit scorecards, fraud scores, and the explanations built on them are logistic regressions or, increasingly, the gradient-boosted trees of Section 6.6.1.

The confusion matrix describes one threshold; the ROC curve describes all of them. Ranking the applicants by $\hat{p}$ gives the order 1, 3, 4, 6, 5, 9, 2, 7, 8, 10, in which the defaulters are 1, 3, 6, and 9. Lowering the threshold past one applicant at a time moves the curve up by 1/4 when that applicant is a defaulter and right by 1/6 when not, tracing the step curve in Figure 6.3. The area under it has an exact probabilistic

Wulin.Suo@Queensu.ca

meaning: it is the probability that a randomly chosen defaulter receives a higher score than a randomly chosen non-defaulter. With four defaulters and six non-defaulters there are $4 \times 6 = 24$ such pairs. Defaulters 1 and 3 outrank all six non-defaulters, defaulter 6 outranks five (all but applicant 4), and defaulter 9 outranks four (all but applicants 4 and 5), so

$$\text{AUC} = \frac{6 + 6 + 5 + 4}{24} = \frac{21}{24} = 0.875,$$

the same number as the area under the step curve, $\frac{1}{6} \times 0.5 + \frac{1}{6} \times 0.75 + \frac{4}{6} \times 1 = 0.875$; a tied pair, had there been one, would count one half. Credit-risk teams often report the equivalent Gini coefficient, or accuracy ratio, $2\,\text{AUC} - 1 = 0.75$, the measure Chapter 9 uses to validate its own scorecard. Because the AUC depends only on the ranking, any increasing transformation of the scores leaves it unchanged. That is both its strength, a threshold-free measure of discrimination, and its blind spot: it says nothing about whether $\hat{p} = 0.585$ really means a 58.5% chance of default, the calibration question taken up in Section 6.4.4.

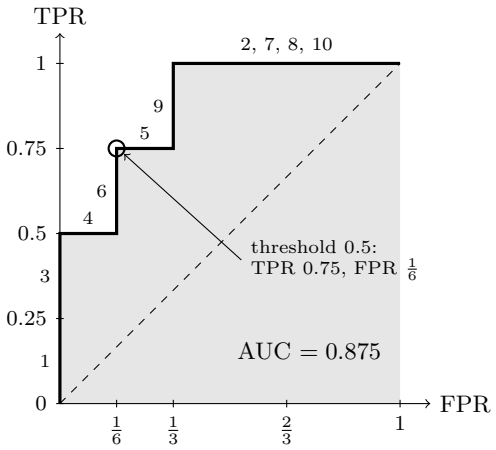


Figure 6.3: ROC curve of the fitted logistic regression for the 10 applicants of Table 6.3. Each step passes one applicant, labeled by number, in order of decreasing $\hat{p}$; the shaded area under the step curve is the AUC, the circle marks the 0.5 threshold of Table 6.4, and the dashed diagonal is the ROC curve of a random ranking.

6.3.5 K-Nearest Neighbors: A Non-Parametric Classifier

Logistic regression, like linear regression, is a parametric model: it assumes a specific functional form (a linear combination of features passed through a sigmoid) and estimates a fixed, finite set of parameters. K-nearest neighbors (KNN) is nonparametric: it assumes no functional form and classifies a new observation by a majority vote among the k most similar observations in the training data.

Consider a small loan-underwriting dataset with two features, debt-to-income ratio (%) and years of credit history, shown in Table 6.5.

To classify a new applicant with debt-to-income of 28% and 4 years of credit history, KNN computes the Euclidean distance from this query point to every training point,

$$d = \sqrt{(28 - x_1)^2 + (4 - x_2)^2},$$

giving

$$d_E = 2.24, \qquad d_A = 7.28, \qquad d_B = 8.94,$$
$$d_F = 10.44, \qquad d_C = 12.37, \qquad d_D = 14.32,$$

sorted from nearest to farthest. With $k = 3$, the three nearest neighbors are E (Default), A (Default), and B (No default), so the majority vote (2 Default, 1 No default) classifies the new applicant as a predicted default.

Wulin.Suo@Queensu.ca

Applicant	Debt/income (%)	Credit history (yrs)	Outcome
A	35	2	Default
B	20	8	No default
C	40	1	Default
D	15	10	No default
E	30	3	Default
F	18	7	No default

Table 6.5: A small loan dataset for K-nearest neighbors classification

Figure 6.4 shows the geometry: the query sits closest to a mix of Default and No-default points, and only its k nearest neighbors, not a global decision boundary, determine the prediction.

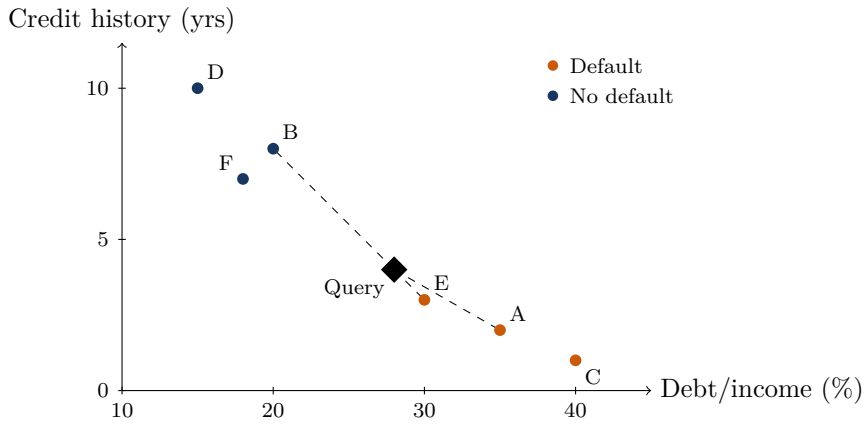


Figure 6.4: The K-nearest neighbors example of Table 6.5: six labeled training points and the query applicant (debt/income 28%, 4 years of credit history, black diamond), with dashed lines to its three nearest neighbors under $k = 3$

The choice of k controls a bias-variance trade-off like the one in Section 6.4.1. A small k (even $k = 1$) makes the prediction sensitive to individual nearby points, including noisy or mislabeled ones (high variance). A large k smooths over local structure and can underfit; when k equals the whole training set, it always predicts the majority class (high bias). KNN also degrades in high dimensions: as the number of features grows, all points become roughly equidistant, the curse of dimensionality, which is one reason KNN is used with a few carefully chosen features rather than on wide financial datasets.

6.3.6 Support Vector Machines

Suppose a credit analyst has a standardized measure of financial distress for a handful of firms and wants the dividing line between later defaulters and survivors that is least likely to misclassify a new firm near the boundary, not just any line that separates the historical data. A support vector machine (SVM) approaches classification geometrically: rather than modeling a probability (logistic regression) or voting among neighbors (KNN), it searches for the hyperplane (a line, in two dimensions) that separates the classes with the widest margin, the distance from the boundary to the nearest training point of either class. A wider margin makes the boundary less sensitive to small perturbations in the data, one reason SVMs often generalize well.

The training points on the margin, closest to the boundary, are called support vectors because they alone determine the hyperplane; every other point could be moved or removed without changing it, unlike linear regression, where every point contributes. When the classes cannot be separated perfectly, a soft-margin SVM lets some points fall on the wrong side of the margin, with a penalty parameter trading margin width against misclassified training points, analogous to the regularization strength λ in ridge and lasso regression.

Wulin.Suo@Queensu.ca

SVMs can also be extended to nonlinear decision boundaries using the "kernel trick," which implicitly maps the features into a higher-dimensional space where a linear separator corresponds to a nonlinear boundary in the original space, without computing the higher-dimensional coordinates. In finance, SVMs have been used for bankruptcy prediction and directional price forecasting, though tree-based ensembles (Section 6.6.1) and neural networks are now more common for large tabular datasets, partly because SVMs scale poorly to very large training sets and trees need less feature preprocessing.

A simple example makes the maximum-margin idea concrete without the quadratic-programming machinery SVMs use in practice. Suppose four training points are perfectly separable along a single feature (say, a standardized measure of financial distress): two "high-risk" points at $x = 1$ and $x = 2$, and two "low-risk" points at $x = 4$ and $x = 5$. Because the classes are perfectly separated along this dimension, the maximum-margin hyperplane is the point halfway between the two closest points of opposite classes, $x = 2$ and $x = 4$, namely $x = 3$.

Writing the decision function as $wx + b$ with $w = 1$, the boundary condition $wx + b = 0$ at $x = 3$ requires $b = -3$, and checking the two nearest points confirms $w(2) + b = 2 - 3 = -1$ and $w(4) + b = 4 - 3 = 1$, the ± 1 margin boundaries the SVM optimization enforces. The points at $x = 2$ and $x = 4$ are the support vectors: they alone determine w and b, and the geometric margin, the distance from the boundary to either margin boundary, is $1/|w| = 1$, for a total margin width of 2, the full gap between the nearest opposite-class points.

The extreme points, $x = 1$ and $x = 5$, play no role in the boundary: moving the point at $x = 1$ to $x = -100$ would not change the hyperplane, whereas moving the support vector at $x = 2$ even slightly would shift it. Figure 6.5 shows the geometry.

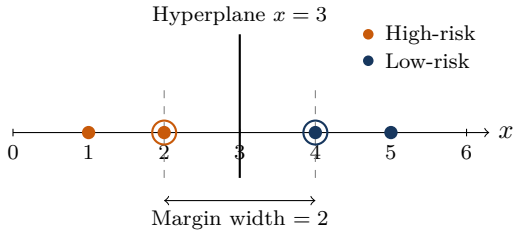


Figure 6.5: The maximum-margin hyperplane for the one-dimensional example: two high-risk points ($x = 1, 2$) and two low-risk points ($x = 4, 5$), with the support vectors circled

6.4 Generalization and Model Selection

A model that fits its training data perfectly has usually learned the noise; this part covers how that failure is detected, measured, and controlled.

6.4.1 Overfitting and Generalization

A central problem in machine learning is overfitting: a complex model fits the training data well yet fails on new data, because it has learned noise and quirks that do not recur outside the sample. Finance is especially exposed, since historical data are noisy and market regimes shift. Table 6.6 shows the pattern in a simulation. The true relationship is a gently curved quadratic, $y = 3 + 2x - 0.5x^2 + \varepsilon$, with x uniform on $[-3, 3]$ and noise ε of standard deviation 3, so no model can achieve a test MSE below the irreducible $\sigma^2 = 9$. Polynomials of increasing degree are fit by least squares to 25 training points and scored on 200 fresh test points, and the whole experiment is repeated 1,000 times on new, seeded samples; the table reports the median of each error across the repetitions. As the model grows more flexible, training error keeps falling, but test error falls only up to a point and then rises.

The minimum test error in Table 6.6 occurs at degree 2, the true degree of the relationship, not at the most flexible model, degree 9, even though degree 9 has the lowest training error in every one of the 1,000 repetitions. Degree 9 also has a higher test error than degree 2 in 99.8% of them, and its errors are erratic as well as large: its mean test MSE across the repetitions is about 9,722, against a median of 27.53, pulled up

Wulin.Suo@Queensu.ca

Polynomial degree	1	2	3	5	9
Training error (MSE)	9.62	7.68	7.33	6.61	5.14
Test error (MSE)	11.57	10.07	10.60	12.05	27.53

Table 6.6: Median training and test error over 1,000 simulated samples as polynomial degree increases

by a few samples in which the fitted polynomial swings wildly between training points. This gap between training and test performance is the signature of overfitting, and the reason a model should never be selected on training performance. Figure 6.6 plots the two rows of Table 6.6 as curves.

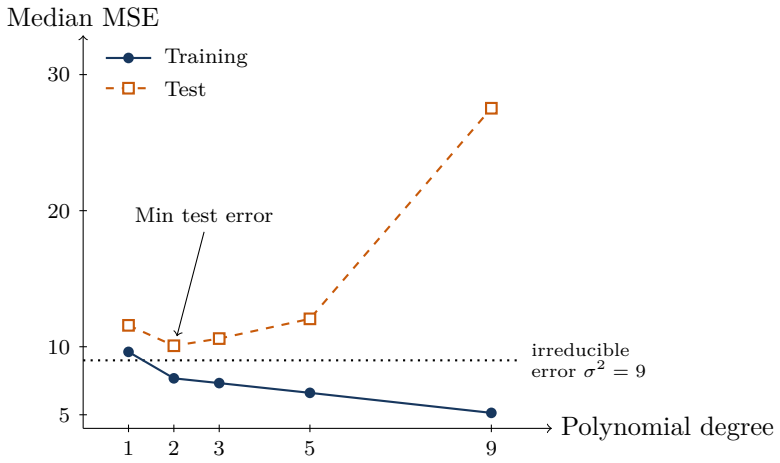


Figure 6.6: Median training and test error (MSE) from Table 6.6 as polynomial degree increases. The dotted line marks the irreducible error, and the vertical axis starts at 4.

Generalization is a model's ability to perform on unseen data, and the goal of model selection is a model that performs well out of sample, not on the training set. k-fold cross-validation formalizes this: the data are split into k roughly equal parts, and the model is trained on $k-1$ of them and evaluated on the remaining part, repeating k times so that every observation is used for validation exactly once. Figure 6.7 illustrates a single fold of 5-fold cross-validation.



Fold 3 held out for validation; Folds 1, 2, 4, 5 used for training

Figure 6.7: One iteration of 5-fold cross-validation. The procedure repeats five times, holding out a different fold each time, and the five validation scores are averaged.

For financial time series, ordinary k-fold cross-validation can mislead because it may train on the future and validate on the past, which is impossible in deployment. Walk-forward validation, in which the training window always precedes the validation window, is the standard alternative, discussed in Chapter 7.

Two subtler leaks survive even when each fold is a contiguous block of time, both from labels that span an interval. A label such as "did this loan default within 12 months" or "the stock's return over the next 20 trading days" depends on data from a window after the observation date. Training observations just before a validation block therefore have labels that overlap it, and those just after it may carry features computed from data inside it. *Purging* removes from the training set every observation whose label window

Wulin.Suo@Queensu.ca

overlaps the validation block, and an *embargo* also drops a short stretch of observations immediately after the block, long enough for serial dependence to fade (López de Prado 2018). With daily observations and 20-day forward-return labels, for instance, the 20 trading days before each validation block are purged and an embargo of similar length follows it. The cost is a somewhat smaller training set; the benefit is a validation score not inflated by the overlap, while training data from both sides of the validation block can still be used. Walk-forward validation avoids the backward leak by construction, and Chapter 13 meets the same overlap problem in backtests of trading signals.

Regularization reduces overfitting by penalizing model complexity in the optimization objective. Ridge regression adds a penalty proportional to the sum of squared coefficients,

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2, \tag{6.6}$$

while lasso regression uses the sum of absolute values,

$$\hat{\beta}^{\text{lasso}} = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \lambda \sum_{j=1}^{p} |\beta_j|. \tag{6.7}$$

In both cases, $\lambda \geq 0$ controls the strength of the penalty: $\lambda = 0$ recovers ordinary least squares, and larger λ shrinks coefficients toward zero, trading a small increase in bias for a potentially large reduction in variance. Lasso can also shrink coefficients exactly to zero, which makes it useful for feature selection when many candidate predictors are available, as is common in finance.

Worked Example 6.4: Ridge versus Lasso on a Genuine and a Spurious Feature

Sparsity is easiest to see with two features, one predictive and one pure noise, fit with both penalties at the same λ. Simulating 200 observations with $y = 2.0\,x_1 + \varepsilon$, where x_1 and x_2 are independent standard-normal features and x_2 is unrelated to y, ordinary least squares identifies both roles, $\hat{\beta}_1^{\text{OLS}} \approx 1.949$ and $\hat{\beta}_2^{\text{OLS}} \approx 0.031$, the latter attributable to sampling noise.

Before fitting, one practical trap. The library used in the companion notebook, scikit-learn, does not scale the two objectives the same way. Its `Ridge` minimizes exactly the ridge objective above, but its `Lasso` minimizes

$$\frac{1}{2n} \sum_{i=1}^{n} (y_i - x_i^\top \beta)^2 + \alpha \sum_{j=1}^{p} |\beta_j|,$$

so `Lasso(alpha=1.0)` is the lasso above with $\lambda = 2n\alpha = 400$, not $\lambda = 1$. Calling both with the default `alpha=1.0` therefore compares a lasso penalty 400 times stronger than the ridge one. Under the single objective written above, the comparison is

$$\begin{aligned} \lambda = 1: &\quad \hat{\beta}^{\text{ridge}} = (1.936,\ 0.030), &\quad \hat{\beta}^{\text{lasso}} = (1.945,\ 0.028), \\ \lambda = 400: &\quad \hat{\beta}^{\text{ridge}} = (0.543,\ -0.019), &\quad \hat{\beta}^{\text{lasso}} = (0.654,\ 0.000). \end{aligned}$$

At $\lambda = 1$ neither penalty does much, since λ is tiny next to a residual sum of squares built from 200 observations. At $\lambda = 400$ the contrast is striking: ridge shrinks both coefficients smoothly, the real one to 0.543, but leaves the spurious one nonzero, while lasso drives the spurious coefficient to *exactly* zero and keeps more of the real one (0.654). Lasso's penalty has a kink at zero that makes it optimal, past a threshold, to set a weak coefficient to zero rather than merely small, so feature selection happens as a side effect of fitting. The scaling trap is a lesson of its own: libraries scale penalties differently (by $1/n$, $1/(2n)$, or not at all), so a nominal penalty value means nothing until you have read the objective the library minimizes, and λ should be tuned by cross-validation rather than copied from a default.

6.4.2 Bias, Variance, and the Trade-off

Suppose two analysts fit different models to the same noisy sample: one a straight line, the other a high-degree polynomial that bends to follow the training points, like the degree-9 fits in Table 6.6. Which should be

Wulin.Suo@Queensu.ca

trusted on new data, and why? The bias-variance trade-off answers this. For a model $\hat{f}$ trained on random training data and evaluated at a point x with true relationship

$$y = f(x) + \varepsilon,$$

where ε has mean zero and variance σ^2, the expected squared prediction error decomposes as

$$\mathbb{E}\left[(y - \hat{f}(x))^2\right] = \underbrace{\Big(f(x) - \mathbb{E}[\hat{f}(x)]\Big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}\big(\hat{f}(x)\big)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible error}}. \tag{6.8}$$

A model with high bias is too simple and systematically misses the true relationship however much data it sees, like a straight line fit to a strongly curved relationship. A model with high variance is too flexible and reacts to the noise in its particular training sample, like the degree-9 polynomial in Table 6.6. A good model balances the two, and the minimum of the test error in Table 6.6 marks that balance: the complexity at which bias and variance are jointly minimized, not the one that minimizes bias alone.

The trade-off matters especially in finance. With limited data and unstable relationships, simpler, higher-bias models can be surprisingly effective, because their lower variance makes them more robust across regimes. With large, high-quality datasets and a stable signal, more complex models may do better. Either way, models should be evaluated out of sample or walk-forward, without assuming that complexity improves results.

6.4.3 Hyperparameter Tuning and Model Selection

Every model introduced in this chapter has at least one hyperparameter, a setting chosen before training rather than learned from the data, that controls its position on the bias-variance spectrum: the ridge penalty λ, the number of neighbors k in KNN, the maximum depth of a decision tree, or the number of trees and learning rate in a gradient-boosted ensemble. Choosing them is often as consequential as choosing the model family, and it must be done without looking at the final test set, or the performance estimate no longer measures generalization.

The standard workflow, called grid search combined with cross-validation, proceeds as follows: specify a grid of candidate hyperparameter values (for example, $\lambda \in \{0.01, 0.1, 1, 10, 100\}$), and for each candidate value, run k-fold cross-validation (or, for time series data, the walk-forward validation introduced in Chapter 7) entirely within the training set, recording the average validation performance. The value with the best average cross-validated performance is selected, and only then is the model retrained on the full training set and evaluated once on the held-out test set, which the search never touched. This nesting is essential: using the test set to choose hyperparameters, even informally by trying a few values and keeping the best, reintroduces the overfitting that cross-validation was designed to prevent and inflates the reported performance.

Financial applications add two complications. First, because financial time series are non-stationary, hyperparameters selected on one period need not remain optimal later, which argues for re-running the tuning periodically. Second, the grid search is itself a form of multiple testing, like the backtest-overfitting concern of Chapter 7: a very large grid raises the chance of a setting that does well on the validation folds by luck, so practitioners often prefer a modest, economically motivated grid to an exhaustive search over hundreds of values.

6.4.4 Model Evaluation

Evaluating a model takes more than a single number, and different tasks need different metrics. For regression, common metrics include mean squared error (MSE), mean absolute error (MAE), and R^2, all illustrated in Section 6.3.1. For classification, common metrics include accuracy, precision, recall, F1, and AUC, all illustrated in Section 6.3.4. In finance one also cares about calibration (do predicted probabilities match observed frequencies), stability (does performance hold up across periods), and economic usefulness (does the output lead to a profitable or risk-reducing decision after costs).

A default model with high accuracy may still be impractical if it is poorly calibrated, produces too many false positives, or fails to separate risk levels usefully. A trading model may perform well statistically but poorly economically once transaction costs and execution limits are counted. Evaluation in finance must therefore be tied to the decision problem, not to a generic statistical scorecard.

Wulin.Suo@Queensu.ca

Calibration is easy to confuse with accuracy. Suppose a credit model assigns a default probability of 20% to a group of 50 borrowers. It is well calibrated for this group if close to 10 of the 50 (that is, $20\% \times 50$) default. If only 2 default, its probabilities are too high even if it still ranks borrowers correctly from most to least risky, a distinction that accuracy and even AUC miss, since both depend only on the ranking. Calibration matters in finance because predicted probabilities feed downstream calculations, such as the expected-loss and reserve calculations of Chapter 9, where an overstated or understated default probability becomes an overstated or understated dollar reserve.

Worked Example 6.5: Measuring and Repairing Calibration

A single number that rewards calibration is the Brier score (Brier 1950), the mean squared difference between predicted probabilities and outcomes,

$$\mathrm{BS} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)^2,$$

which is zero only for perfect, certain forecasts and, unlike the AUC, penalizes a probability that is too high or too low even when the ranking is right. For the 50 borrowers above, the 20% forecast scores $\frac{1}{50}\left[10\times0.8^2+40\times0.2^2\right] = 0.160$ when 10 of them default. When only 2 default it scores $\frac{1}{50}\left[2\times0.8^2+48\times0.2^2\right] = 0.064$, while the calibrated forecast of 4% would have scored 0.038: the same borrowers and the same ranking, and a score 40% lower for getting the level right. (The two 20% scores, 0.160 and 0.064, are not comparable with each other, since they grade forecasts of different outcomes; Brier scores rank competing forecasts of the same outcomes.)

A model that ranks well but is miscalibrated need not be rebuilt; its scores can be remapped. **Platt scaling** (Platt 1999) fits a one-feature logistic regression of the outcome on the score's log-odds, $\Pr(y = 1 \mid s) = 1/\left(1+e^{-(a+b\,\mathrm{logit}(s))}\right)$ with $\mathrm{logit}(s) = \log\left(s/(1-s)\right)$, on a held-out calibration set. **Isotonic regression** (Zadrozny and Elkan 2002) instead fits the best non-decreasing step function from score to probability, which assumes no functional form but needs more data. Both preserve the order of the scores (isotonic regression up to ties), so they repair calibration without changing discrimination.

To see both at work, simulate borrowers whose true default probability is $1/(1+e^{-(-3+z)})$ for a standard-normal risk factor z, scored by a model with $s = 1/(1+e^{-(-1.8+1.6z)})$ that ranks them perfectly but overstates almost every probability, as a model trained in a period of high defaults might. On 20,000 test borrowers the default rate is 6.98% while the score averages 22.30%, and the 3,122 borrowers scored between 15% and 25% average a score of 19.62% but default at only 6.18%, the 20%-versus-4% situation above at scale. Fitting each remedy on a separate calibration set of 2,000 borrowers gives Table 6.7. Platt scaling, with fitted $a = -1.921$ and $b = 0.655$, brings the Brier score from 0.1053 to 0.0593, matching the true probabilities, because the miscalibration here has the logistic form Platt scaling assumes (the true map is $a = -1.875$, $b = 0.625$). Isotonic regression gets almost as far, to 0.0595, without assuming a form, at the price of a slightly lower AUC, since its flat steps tie borrowers the raw score had separated.

Forecast	Mean forecast	Brier score	AUC	Mean forecast, 15–25% raw band
Raw model score	22.30%	0.1053	0.7672	19.62%
Platt scaling	6.64%	0.0593	0.7672	5.49%
Isotonic regression	6.69%	0.0595	0.7641	3.89%
True probability	7.01%	0.0593	0.7672	5.97%
Observed default rate	6.98%			6.18%

Table 6.7: Recalibrating an overconfident default model on 2,000 borrowers, scored on 20,000 fresh borrowers

Two cautions apply. The calibration set must be separate from the training data, or the remap learns the model's in-sample overconfidence instead of correcting it. And a remap fitted in one credit environment goes stale in the next, so recalibration is part of monitoring rather than a one-time fix. Niculescu-Mizil and Caruana (2005) compare the two methods across many model families; Chapter 21 applies isotonic recalibration to a real credit-card default model.

Wulin.Suo@Queensu.ca

6.4.5 Quantifying Uncertainty: Split Conformal Prediction

Every model so far returns a single number: a fitted spread of $20 + 47(4.5) = 231.5$ basis points for an issuer at 4.5 turns of leverage, a default probability, a class label. A risk manager setting a limit, a trader quoting a price, or a credit officer deciding whether a case needs a second look usually needs a range as well: how far from the point forecast could the truth plausibly be? Section 6.3.2's standard errors answer a different question, how precisely the *coefficients* are pinned down. The classical prediction interval for a new observation relies on normal, constant-variance errors and a correctly specified model, and for a random forest, a gradient-boosted ensemble, or a neural network there is no such formula.

Conformal prediction (Vovk et al. 2022) puts an interval around *any* fitted model, with a coverage guarantee that needs no distributional assumptions and holds in finite samples. Its simplest and most widely used form, **split conformal prediction** (Lei et al. 2018), takes four steps:

1. Split the available data into a *proper training set* and a *calibration set* of n observations that the model never sees during fitting.
2. Fit any model $\hat{f}$ on the proper training set.
3. On the calibration set, compute a *nonconformity score* for each observation, which for regression is usually the absolute residual $s_i = |y_i - \hat{f}(x_i)|$.
4. Set $\hat{q}$ to the k-th smallest calibration score, where

$$k = \lceil (n+1)(1-\alpha) \rceil, \qquad \hat{q} = s_{(k)}, \tag{6.9}$$

   and report the interval $C(x) = \left[\hat{f}(x) - \hat{q},\ \hat{f}(x) + \hat{q}\right]$ for each new case.

Equivalently, $\hat{q}$ is the empirical quantile of the calibration scores at level $\lceil (n+1)(1-\alpha) \rceil / n$, slightly above the nominal $1-\alpha$; the $(n+1)$ in place of n is a small-sample correction whose purpose is explained below. If $k > n$, which happens when $n < 1/\alpha - 1$, the calibration set is too small to support the requested coverage and the interval is the whole real line.

The guarantee is that, provided the calibration observations and the new observation (X_{n+1}, Y_{n+1}) are **exchangeable**, meaning that their joint distribution is unchanged by reordering them (independent draws from one distribution are the leading case),

$$1 - \alpha \ \le\ \Pr\left(Y_{n+1} \in C(X_{n+1})\right) \ \le\ 1 - \alpha + \frac{1}{n+1}, \tag{6.10}$$

where the upper bound requires only that the scores have no ties. The argument is a counting one. Exchangeability makes the new observation's score s_{n+1} equally likely to occupy any of the $n+1$ rank positions among all $n+1$ scores, so the probability that it lands at or below the k-th smallest calibration score is exactly $k/(n+1)$, and choosing $k = \lceil (n+1)(1-\alpha) \rceil$ makes $k/(n+1) \ge 1-\alpha$. Nothing in the argument depends on what $\hat{f}$ is, how well it fits, or the shape of the error distribution: a poor model produces large scores and wide intervals, but they still cover at the promised rate. The guarantee is *marginal*: the probability is over the random calibration set and new case together, so it promises 90% coverage on average, not 90% given one particular calibration set, and not 90% within every subgroup of borrowers or issuers.

Worked Example 6.6: Split Conformal Intervals for Credit Spreads

To see every number, simulate issuers from a relationship close to Section 6.3.1's fitted line, spread $= 20 + 47 \times$ leverage $+ \varepsilon$, with leverage drawn uniformly between 1 and 7 turns and ε normal with a standard deviation of 25 basis points. OLS on a proper training set of 40 issuers gives $\widehat{\text{spread}} = 9.91 + 50.12 \times$ leverage. A separate calibration set of $n = 24$ issuers then produces the 24 absolute residuals in Table 6.8, sorted from smallest to largest.

For a 90% interval, $\alpha = 0.10$, so Equation (6.9) gives $(n+1)(1-\alpha) = 25 \times 0.9 = 22.5$, rounded up to $k = 23$, and the conformal quantile is the 23rd smallest score, $\hat{q} = 44.12$ basis points (printed as 44.1 in the table). A new issuer at 4.5 turns of leverage receives a point forecast of $9.912 + 50.122 \times 4.5 \approx 235.46$ basis

Wulin.Suo@Queensu.ca

Rank (i) and score $s_{(i)}$, bps							
(1) 1.2	(2) 4.5	(3) 4.9	(4) 9.7	(5) 11.4	(6) 14.7	(7) 15.6	(8) 16.4
(9) 17.2	(10) 20.3	(11) 20.6	(12) 21.4	(13) 23.9	(14) 25.0	(15) 27.9	(16) 28.8
(17) 29.5	(18) 29.5	(19) 34.9	(20) 37.6	(21) 37.9	(22) 38.9	(23) 44.1	(24) 44.3

Table 6.8: Sorted absolute residuals $|y_i - \hat{f}(x_i)|$ of the 24 calibration issuers, in basis points

points (the fitted coefficients to three decimals; the two-decimal values above would give 235.45) and the interval

$$235.46 \pm 44.12 = [191.34,\ 279.58] \text{ basis points.}$$

The width is the same for every issuer, since the score is a plain absolute residual. Because the scores are continuous, Equation (6.10) holds with the exact value $k/(n+1) = 23/25 = 0.920$, inside the guaranteed range $[0.900,\ 0.940]$.

Two checks confirm the theory. On 5,000 fresh test issuers drawn from the same process, 91.52% of the true spreads fall inside their intervals. And repeating the whole calibration step 10,000 times, each time with a new set of 24 issuers and 200 test issuers, gives an average coverage of 91.94%, essentially the predicted 92.0%. The same repetition also shows what "marginal" means in practice: across calibration draws, the coverage achieved by one particular set of 24 issuers ranges from 81.0% at the 5th percentile to 99.0% at the 95th. That spread is predicted too: given one calibration set, coverage follows a $\text{Beta}(k,\ n+1-k) = \text{Beta}(23, 2)$ distribution, whose 5th and 95th percentiles are 81.7% and 98.5%, the small remaining gap reflecting the finite 200-issuer test sets. A larger calibration set tightens this spread without changing the average.

The correction matters. Taking the plain empirical 90th percentile instead, $k = \lceil n(1-\alpha) \rceil = \lceil 21.6 \rceil = 22$, gives $\hat{q} = 38.87$ basis points and an exact coverage of only $22/25 = 0.880$, below the target: 86.66% on the 5,000 test issuers and 87.89% on average over the 10,000 repetitions. With small calibration sets, the extra $+1$ is what turns a nominal 90% interval into a real one.

Limits. Exchangeability is the whole foundation of Equation (6.10), and financial data violate it routinely. Simulating a stress regime in which the noise standard deviation doubles from 25 to 50 basis points, while keeping the same $\hat{q} = 44.12$, cuts the empirical coverage of the nominal 90% interval to 61.04%. That is the situation in which a risk manager most needs the interval to work: the regime shift, volatility clustering, and concept drift discussed in Chapter 7 and Section 6.7. Time-ordered data are not exchangeable even without a regime change, so the calibration set should come from the most recent period and be refreshed as outcomes arrive; adaptive variants that update $\hat{q}$ online as coverage errors accumulate are designed for this setting (Gibbs and Candès 2021). A second limit is that the constant width ignores heteroskedasticity: if errors are larger for highly levered issuers, a single $\hat{q}$ over-covers safe issuers and under-covers risky ones while still averaging 90%. Dividing each residual by a model of its typical size, or calibrating a quantile regression, gives intervals that widen where the model is less certain. Chapter 18's Advanced Topics returns to conformal prediction for classifiers, where the output is a set of labels and a borderline credit applicant whose set contains both "repay" and "default" is flagged for human review (see also Angelopoulos and Bates 2023).

6.4.6 Prediction versus Causation

Every model in this chapter answers a *predictive* question: given what is observed about this borrower or issuer, what outcome should be expected? Many of the decisions they feed ask a *causal* question instead: if the institution changes something, what will happen? Raising a cardholder's credit limit, offering a retention discount, calling a delinquent borrower, or cutting a loan's interest rate are interventions, and choosing among them requires knowing how the outcome responds to the intervention, not how the two moved together in historical data. Even the slope of 47 basis points per turn of leverage in Section 6.3.1 is an association across issuers. Highly levered firms differ from lightly levered ones in industry, asset tangibility, and cash-flow volatility, so one firm adding a turn of leverage need not see its spread rise by 47 basis points. Chapter 7's Granger causality carries the same caveat: it tests whether one series helps *predict* another, not whether it causes it.

Wulin.Suo@Queensu.ca

The potential-outcomes framework makes the distinction precise. Let $D = 1$ if a customer receives an intervention (a *treatment*) and $D = 0$ otherwise, and let $Y(1)$ and $Y(0)$ be the outcomes the same customer would experience with and without it. Only one of the two is ever observed. The *average treatment effect* is $\tau = \mathbb{E}[Y(1) - Y(0)]$. Comparing the treated and untreated groups in observational data gives, when the effect is the same for everyone,

$$\mathbb{E}[Y \mid D = 1] - \mathbb{E}[Y \mid D = 0] = \tau + \underbrace{\mathbb{E}[Y(0) \mid D = 1] - \mathbb{E}[Y(0) \mid D = 0]}_{\text{selection bias}}. \tag{6.11}$$

The selection-bias term is the difference the two groups would show even if nobody were treated. It is nonzero whenever a **confounder**, a variable driving both who is treated and the outcome, differs between the groups. In consumer credit the canonical confounder is creditworthiness. Lenders grant credit-limit increases mainly to customers who look safe, and safe customers default less, so the customers who received increases would have defaulted less *even without* them.

In a linear model the size of the distortion has a simple form. Suppose the outcome is $Y = \beta_0 + \tau D + \gamma Z + \varepsilon$, where Z is the confounder, and an analyst regresses Y on D alone. The "short" regression's coefficient on D is

$$\hat{\beta}_{\text{short}} = \hat{\tau} + \hat{\gamma}\,\hat{\delta}, \tag{6.12}$$

where $\hat{\tau}$ and $\hat{\gamma}$ come from the "long" regression of Y on D and Z, and $\hat{\delta}$ from regressing Z on D, which for a binary D is the gap in the average of Z between the treated and untreated groups. This **omitted-variable bias** formula holds exactly in any sample, not just on average, so it can be checked to the last decimal.

Worked Example 6.7: A Credit-Limit Increase: Naive, Adjusted, and Randomized Estimates

Consider 20,000 cardholders with a creditworthiness score Z distributed uniformly between -1 (weakest) and $+1$ (strongest). The bank grants a limit increase with probability $\Pr(D = 1 \mid Z) = 0.5 + 0.4Z$, so its strongest customers receive one 90% of the time and its weakest 10% of the time. Each customer's probability of defaulting within twelve months is

$$\Pr(\text{default}) = 0.08 + 0.02\,D - 0.06\,Z,$$

which stays between 0.02 and 0.16. By construction, the true causal effect of a limit increase is $\tau = +0.02$: extra credit raises every customer's default probability by two percentage points.

In the simulated observational data, 50.44% of customers receive an increase. Their average creditworthiness is 0.2665, against -0.2662 for the rest, a gap of $\hat{\delta} = 0.5327$. The default rate is 8.25% among customers who received an increase and 9.80% among those who did not, so the naive comparison, identical to the short regression of default on D, says increases *lower* default by 1.55 percentage points. A predictive model would be right to use this pattern: the increase flag is a useful feature, because it carries information about creditworthiness that the bank observed when granting it. As a guide to policy, however, it points the wrong way: a lender that raised limits to reduce defaults would raise them instead.

Table 6.9 collects the estimates. Adding Z to the regression gives $\hat{\tau} = +0.0180$ and $\hat{\gamma} = -0.0629$, close to the true $+0.02$ and -0.06, and Equation (6.12) reconciles the two regressions exactly:

$$\hat{\beta}_{\text{short}} = 0.0180 + (-0.0629)(0.5327) = 0.0180 - 0.0335 = -0.0155.$$

In population terms, the gap in average creditworthiness is 0.5333, so the bias is $-0.06 \times 0.5333 = -0.032$, large enough to turn a true effect of $+0.02$ into an apparent effect of -0.012.

Adjustment works here only because the analyst observes Z exactly. With only a noisy proxy, the true score plus normal noise with a standard deviation of 0.5, the adjusted estimate is $+0.0027$: the regression removes some of the confounding but leaves most of the true effect hidden. In practice the bank's score is at best a proxy for the creditworthiness that drives default, and confounders it never recorded cannot be adjusted for.

A randomized experiment removes the problem by construction. Drawing 20,000 fresh customers and letting a coin flip decide who receives the increase puts 10,013 in the treated group and 9,987 in the control

Wulin.Suo@Queensu.ca

Estimator	Estimated effect of an increase
Naive: default regressed on D alone	-0.0155
Adjusted: default regressed on D and Z	$+0.0180$
Adjusted for a noisy proxy $Z + \mathrm{N}(0, 0.5^2)$	$+0.0027$
Randomized experiment, difference in default rates	$+0.0193$
True effect τ	$+0.0200$

Table 6.9: Four estimates of the effect of a credit-limit increase on the twelve-month default rate, each from 20,000 simulated cardholders

group, with average creditworthiness of -0.0008 and -0.0116, balanced up to chance. Their default rates are 9.94% and 8.01%, a difference of $+0.0193$. With the standard error of a difference in proportions,

$$\sqrt{\frac{0.0994(1-0.0994)}{10{,}013} + \frac{0.0801(1-0.0801)}{9{,}987}} \approx 0.0040,$$

the 95% confidence interval is $[+0.0113,\ +0.0272]$, which contains the true $+0.02$ and excludes both zero and the naive estimate.

Experiments as the gold standard. Randomization makes the treated and control groups alike, on average, in every characteristic, observed or not, so the selection-bias term in Equation (6.11) is zero and a simple difference in means estimates τ. Financial firms run such experiments routinely, as A/B tests of offers, pricing, and collection strategies, and the method has produced some of the cleanest evidence on credit. Aydin (2022) reports a field experiment at a large retail bank in Turkey: of 45,307 customers preapproved for a lender-initiated limit increase, a randomly chosen control group had the increase withheld for nine months. Treated customers spent about 11 cents per dollar of new limit in the quarter of the increase, rising to about 28 cents by the third year, a causal estimate no regression on the bank's historical limit decisions could have delivered. Experiments have costs and limits. The control group forgoes something the lender believes is good for it; consumer-protection and fair-lending rules constrain which terms can be varied and for whom; outcomes such as default take months or years to observe; and many questions, such as the effect of a recession or a central-bank rate move, cannot be randomized. Chapter 19's bandit algorithms address part of this tension: how to keep learning from experiments while limiting the cost of assigning customers to inferior options.

Observational methods. When an experiment is impossible, several methods try to recover causal effects from observational data, each replacing randomization with an assumption that must be argued rather than tested. *Difference-in-differences* compares the change in outcomes over time for a group affected by a policy against the change for an unaffected group, and requires that the two would have followed parallel trends without the policy. *Instrumental variables* exploit a variable that shifts who is treated but affects the outcome only through the treatment, such as an arbitrary change in an approval rule. *Double/debiased machine learning* (Chernozhukov et al. 2018) uses flexible models of the kind developed in this chapter to predict both the outcome and the treatment from a large set of controls, then estimates the effect from the two sets of residuals with cross-fitting, so that regularization does not bias the causal estimate; like the adjusted regression above, it still assumes every important confounder is among the controls. *Uplift modeling* predicts how much an individual customer's outcome would change if treated, so that a retention offer or a collections call goes to the customers it changes rather than those who would have responded anyway; it is most reliable when trained on experimental data. Angrist and Pischke (2009) is the standard practical introduction to these designs.

The lesson for the rest of the book is a division of labor. Predictive models, and the importance measures and Shapley values Chapter 18 uses to explain them, describe associations learned from training data. They are the right tool for ranking applicants, flagging transactions, or forecasting losses, and the wrong one for deciding what a new policy would do. Section 6.7's warning about feedback and reflexivity is the market-level version of the same point: once a model's output is used to intervene, the historical association it learned is no longer a reliable guide.

Wulin.Suo@Queensu.ca

6.5 Working with Financial Data

Financial data breaks several assumptions the previous parts made quietly, and the features handed to a model matter more than the model.

6.5.1 Practical Considerations in Financial ML

Deploying machine learning in finance requires attention to data leakage, sample selection, and concept drift. Data leakage occurs when the model uses information that would not be available at prediction time, such as a variable recorded only after the outcome is known, producing optimistic backtest results that vanish in real time. Sample selection issues arise when the training data are not representative of the future environment, for instance training a credit model only on approved loans, which excludes information about applicants who were rejected. Concept drift occurs when the relationship between features and outcomes changes over time, for example when a recession changes which borrower characteristics matter most for default.

A compact view of the modeling lifecycle is shown in Figure 6.8.

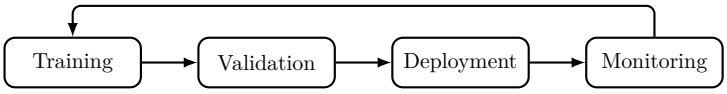


Figure 6.8: A machine learning workflow in finance should include training, validation, deployment, and continuous monitoring, with findings from monitoring feeding back into retraining

These issues often determine whether a model survives in production. Good financial modeling means not only fitting a relationship but making sure it is stable enough to stay useful, and building the monitoring to detect when it stops being useful.

6.5.2 Feature Engineering for Financial Data

Every model in this chapter takes a fixed set of numerical features, but raw financial data (prices, statement line items, transaction records) are rarely usable directly, and transforming them into informative inputs, called feature engineering, often matters as much as the choice of algorithm. Several patterns recur. Ratios, such as the debt-to-equity and margin ratios of Chapter 4, normalize dollar figures so that firms of very different sizes become comparable, as that chapter's common-size statements did.

Lagged and rolling features, such as a 30-day rolling volatility or a 12-month trailing revenue growth rate, summarize recent history into a single number a model can consume, echoing the rolling-window calculations introduced in Chapter 2. Interaction features, such as leverage multiplied by a recession indicator, let a linear model capture an effect of one variable that depends on the level of another, which a plain linear model cannot represent. Categorical features, such as industry sector, are converted to numbers by one-hot encoding (a binary column for each category), so that sector-specific behavior of the kind discussed in Chapter 4 enters the model as a signal rather than an arbitrary label. Table 6.10 and Table 6.11 make these transformations concrete for a small cross-section of three firms.

Firm	Sector	Debt	Equity	Rev.$_t$	Rev.$_{t-4}$	Recession	Mkt Cap
1	Tech	20	80	150	120	0	45.0
2	Energy	90	60	210	230	1	8.0
3	Retail	40	40	95	90	0	1.2

Table 6.10: Raw firm-level inputs for a small cross-section (Debt, Equity, and Revenue in $ millions; Market Cap in $ billions; Recession is 1 if the firm's primary sector was in a broad-based downturn during the period)

Table 6.11 shows the engineered features these raw inputs produce. The leverage ratio and year-over-year revenue growth follow directly from the raw dollar figures; the interaction term is zero for both non-recession

Wulin.Suo@Queensu.ca

firms whatever their leverage, but equals 1.50 for Firm 2, flagging that its elevated leverage coincides with a sector-wide downturn. One-hot encoding converts the raw Sector column into three binary indicators: Firm 1 becomes (Technology=1, Energy=0, Retail=0), Firm 2 becomes (Technology=0, Energy=1, Retail=0), and Firm 3 becomes (Technology=0, Energy=0, Retail=1).

Firm	Leverage	Rev. growth (YoY)	Leverage×Recession	$\log_{10}$(Mkt Cap, \$B)	Mkt Cap pctile
1	0.25	25.0%	0.00	1.65	100%
2	1.50	−8.7%	1.50	0.90	50%
3	1.00	5.6%	0.00	0.08	0%

Table 6.11: Engineered features derived from Table 6.10

Two further considerations are specific to finance. First, every engineered feature must respect point-in-time discipline: a feature computed from data that would not have been available at prediction time introduces data leakage, even when the computation, such as a rolling average, seems innocuous. Second, financial features are frequently skewed or heavy-tailed, and Table 6.11's market-capitalization columns make this concrete: the three firms' raw market caps span more than an order of magnitude (\$1.2 billion to \$45.0 billion, a ratio of 37.5×), while $\log_{10}$ of the same figures compresses this to a range of about 1.57 (0.08 to 1.65), and the cross-sectional percentile rank compresses it further still, to a bounded $[0, 100\%]$ scale regardless of the firms' raw dollar magnitudes. Such transformations make skewed or heavy-tailed features better behaved as inputs to linear and distance-based models.

Worked Example 6.8: Feature Scaling: One Applicant, Three Answers

Distance-based and penalized models need features on a common scale, as the two features of Table 6.5 show. Debt-to-income runs from 15 to 40 percentage points and credit history from 1 to 10 years, so the Euclidean distance of the K-nearest neighbors example is dominated by whichever feature has the larger numerical spread, a choice made by the units rather than the economics. Consider an applicant with a 28% debt-to-income ratio and 7 years of credit history. In the table's units, percentage points and years, the three nearest neighbors are E (distance 4.47), B (8.06), and A (8.60), two defaults and one non-default, so 3-NN predicts default. Record credit history in months instead, which carries the same information, and the three nearest neighbors become F (10.00), B (14.42), and D (38.28), all non-defaults.

Standardization removes the dependence on units by replacing each feature with its z-score,

$$z = \frac{x - \bar{x}}{s_x},$$

where $\bar{x}$ and s_x are the feature's mean and standard deviation in the training data. For the six applicants, debt-to-income has mean 26.33 and standard deviation 9.25, and credit history mean 5.17 and standard deviation 3.34 (population standard deviations, the convention of scikit-learn's `StandardScaler`), so the query applicant becomes (0.180, 0.549). In standardized units the three nearest neighbors are B (0.92), F (1.08), and E (1.22), and the prediction is no default, whether history was recorded in years or in months. Neither raw answer was wrong by the algorithm's own logic; each let the units decide how much each feature counted.

The logic reaches beyond KNN. Ridge and lasso penalize $\sum_j \beta_j^2$ or $\sum_j |\beta_j|$, and a coefficient's size depends on its feature's units, so an unscaled penalty shrinks some features more than others for no economic reason. Gradient descent converges faster when features share a scale, because the loss surface is less elongated, and SVMs and neural networks are similarly sensitive. Tree-based models, by contrast, are unaffected, since a split on $x > t$ separates the same observations however x is measured. Two rules follow. Means and standard deviations must be estimated on the training data only and applied unchanged to validation and test data, since computing them on the full sample is a mild form of data leakage. And heavy-tailed features are usually transformed first, with the logarithms or percentile ranks of Table 6.11, since a few extreme values inflate the standard deviation and squash everyone else's z-scores toward zero.

Wulin.Suo@Queensu.ca

6.6 Beyond Linear Models

This part covers the model families that dominate practice when the relationship is not linear and the data are not tidy.

6.6.1 Tree-Based Models and Ensembles

Tree-based methods are among the most useful models in applied finance because they capture nonlinear relationships and interactions without the analyst specifying them in advance. A decision tree splits the data recursively according to feature thresholds, choosing at each step the split that most reduces impurity in the resulting groups. For classification, a common impurity measure is the Gini impurity,

$$G = 1 - \sum_{k=1}^{K} p_k^2,$$

where p_k is the proportion of observations in a node belonging to class k; $G = 0$ when a node is pure (all one class) and is largest when classes are evenly mixed. An alternative is entropy, $H = -\sum_k p_k \log_2 p_k$, which behaves similarly.

Worked Example 6.9: One Split of a Decision Tree

Growing a tree on the ten applicants of Table 6.3, with debt-to-income as the only feature, shows how a split is chosen. The root node holds four defaults and six non-defaults, so its impurity is $G = 1 - (0.4^2 + 0.6^2) = 0.48$. A candidate split sends applicants with a ratio above a threshold t to one child node and the rest to the other, and is scored by the two children's impurities weighted by their sizes. Only the midpoints between consecutive observed ratios need to be tried, nine in all. Splitting at $t = 35$, close to the logistic regression's 50% boundary of 35.33%, puts applicants 1, 3, 4, and 6 (three defaults out of four) above and the other six (one default) below:

$$G_{\text{above}} = 1 - (0.75^2 + 0.25^2) = 0.375, \quad G_{\text{below}} = 1 - \left((\tfrac{1}{6})^2 + (\tfrac{5}{6})^2\right) = 0.278, \quad \tfrac{4}{10} \times 0.375 + \tfrac{6}{10} \times 0.278 = 0.317.$$

Splitting at $t = 28.5$ instead puts all four defaults and two non-defaults (applicants 4 and 5) above, and four non-defaults below:

$$G_{\text{above}} = 1 - \left((\tfrac{4}{6})^2 + (\tfrac{2}{6})^2\right) = 0.444, \quad G_{\text{below}} = 0, \quad \tfrac{6}{10} \times 0.444 + \tfrac{4}{10} \times 0 = 0.267.$$

Checking all nine thresholds confirms that $t = 28.5$ has the lowest weighted impurity, a reduction of $0.48 - 0.267 = 0.213$ from the root, and it is the split scikit-learn's `DecisionTreeClassifier` chooses. The tree and the logistic regression draw their lines in different places because they optimize different things: the Gini criterion rewards the pure node of four safe applicants, while the logistic regression's 50% boundary is a by-product of fitting probabilities to every applicant. A full tree would continue by splitting each child in the same way, trying every feature at every node, until a stopping rule such as a maximum depth or a minimum node size halts it.

A single tree tends to have high variance: small changes in the training data can produce very different splits. Random forests reduce this variance by averaging many trees, each trained on a bootstrap sample of the data with a random subset of features at each split, so that the trees are decorrelated. Gradient boosting instead builds trees sequentially, each one correcting the errors of the ensemble built so far. Write $F_m(x)$ for the ensemble's prediction after m trees have been added; boosting initializes $F_0(x)$ to a simple constant (typically the mean of the target for regression, or the log-odds of the positive class for classification) and then grows the ensemble one shallow tree at a time,

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad m = 1, \ldots, M, \tag{6.13}$$

where h_m is a new regression tree fit not to the target y but to the current pseudo-residuals, $y_i - F_{m-1}(x_i)$ for squared-error loss. More generally these are the negative gradient of the loss with respect to the current predictions $F_{m-1}(x_i)$, hence the name "gradient" boosting, so each new tree targets what the ensemble still gets wrong.

Wulin.Suo@Queensu.ca

The learning rate $\nu \in (0, 1]$, typically well below 1 (commonly 0.01 to 0.3), shrinks each tree's contribution: a smaller ν needs more trees M to reach a given fit but tends to generalize better, the same bias-variance trade-off as the ridge and lasso penalties. Figure 6.9 shows the sequential structure. Each stage cannot be fit until the previous stage's pseudo-residuals are known, which is why random forests parallelize trivially across trees and gradient boosting does not. Rather than reducing variance by averaging, boosting reduces bias by correcting the current model's errors, and it is combined with a small learning rate and a limited number of trees to control overfitting.

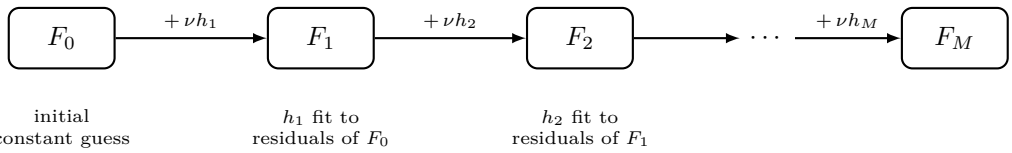


Figure 6.9: Gradient boosting as a sequential chain of shrunken trees, each fit to correct the previous stage's residuals

In finance such models are widely used for default prediction, credit scoring, and fraud detection, and they can estimate which features drive a prediction. They are not automatically more reliable than simpler models: a tree-based model can still overfit, especially on noisy data or when cross-validation ignores the time structure, so validation along the lines of Section 6.4.1, with walk-forward schemes rather than random splits, remains essential.

Table 6.12 summarizes the two ensembling strategies, which are easily confused although they solve different problems.

	Bagging (e.g., random forest)	Boosting (e.g., gradient boosting)
Trees built	In parallel, independently	Sequentially, each correcting the last
Primarily reduces	Variance	Bias
Base learner	Deep, low-bias trees	Shallow, high-bias trees
Overfitting risk	Lower, more robust to noise	Higher, requires careful tuning
Typical use case	Robust baseline with minimal tuning	Squeezing out maximum predictive accuracy

Table 6.12: Bagging (random forests) versus boosting (gradient-boosted trees)

Neither approach dominates, but the difference in overfitting risk matters in finance: a gradient-boosted model with many trees and a small learning rate can reach very low training error, and the walk-forward validation of Chapter 7, rather than a random split, is what catches overfitting before it reaches production.

6.6.2 Naive Bayes: A Probabilistic Classifier

Chapter 1 introduced Bayes' theorem with a fraud-detection example; naive Bayes turns it into a classification algorithm. Given features $x_1, \ldots, x_p$, it estimates the probability of each class k using Bayes' theorem,

$$\Pr(y = k \mid x_1, \ldots, x_p) \propto \Pr(y = k) \prod_{j=1}^{p} \Pr(x_j \mid y = k),$$

and predicts whichever class has the highest resulting probability. The "naive" assumption is that the features are conditionally independent given the class. It is rarely true (a firm's leverage and interest coverage are clearly related) but often good enough for a fast, effective baseline, particularly for text classification such as the sentiment analysis of Chapter 16, where the features are word counts and the false independence assumption distorts less than it would for tightly coupled financial ratios. Naive Bayes trains almost instantly

Wulin.Suo@Queensu.ca

even on large datasets, since fitting means estimating frequencies rather than solving an optimization problem, which makes it a useful baseline before investing in a more sophisticated model.

For text, where each feature x_j counts how often a word appears, $\Pr(x_j \mid y = k)$ is naturally estimated from training-data frequencies, the fraction of class k's total word count taken by word x_j. This raw estimate has a serious flaw: a word that never appears in class k's training documents gets probability zero, and because the naive Bayes score is a *product* across features, one zero forces the whole product to zero however strongly every other word favors that class, an artifact of a small sample rather than evidence that the class is impossible. Laplace smoothing (also called add-one smoothing) fixes this by adding a small constant to every count before normalizing,

$$\Pr(x_j \mid y = k) = \frac{\text{count}(x_j, k) + 1}{\text{count}(k) + V},$$

where $\text{count}(x_j, k)$ is how many times word x_j appeared across class k's training documents, $\text{count}(k)$ is the total word count across those same documents, and V is the vocabulary size, the number of distinct words being counted. Adding one to the numerator gives every word a positive probability even if it never appeared in that class, while adding V to the denominator keeps the probabilities summing to one across the vocabulary. Chapter 16's 10-K risk-factor classifier applies this correction.

6.6.3 Clustering and Dimensionality Reduction

Not every task predicts a known outcome; sometimes the goal is to find structure in a large set of assets or transactions. Suppose a portfolio manager has a few hundred holdings and no label saying which behave similarly, and wants an algorithm that groups them from their measured characteristics rather than by eye. K-means clustering partitions observations into K groups by minimizing the within-cluster sum of squared distances to each group's center,

$$\arg\min_{\{S_k\}} \sum_{k=1}^{K} \sum_{x_i \in S_k} \|x_i - \mu_k\|^2, \qquad \mu_k = \frac{1}{|S_k|} \sum_{x_i \in S_k} x_i,$$

where S_k denotes the set of observations assigned to cluster k and μ_k is that cluster's center, the mean of its assigned points. This groups similar observations together, for example firms with similar financial profiles or trading accounts with similar behavior. Principal component analysis (PCA) is the standard dimensionality-reduction method: it finds the direction w (with $\|w\| = 1$) that maximizes the variance of the projected data, $\arg\max_w \text{Var}(Xw)$, then finds the next such direction orthogonal to the first, and so on. In finance, PCA applied to a panel of asset returns often reveals that a small number of components, frequently interpretable as a broad market factor followed by sector or style factors, explain the large majority of the total variance.

These approaches help uncover latent structure in high-dimensional data. A portfolio manager may want to know whether returns are driven by a few common factors or many idiosyncratic sources, a question taken up in Chapter 12; a fraud analyst may want clusters of unusual behavior that deserve investigation. In both cases unsupervised learning is a useful first step before more targeted modeling.

To make K-means concrete, consider four firms described by two features, leverage and profit margin (both scaled to lie between 0 and 1):

$$F_1 = (0.20, 0.30), \qquad F_2 = (0.30, 0.25),$$
$$F_3 = (0.80, 0.05), \qquad F_4 = (0.90, 0.10).$$

Initializing two cluster centers at F_1 and F_4, the algorithm alternates between two steps: assign each point to its nearest center, then recompute each center as the mean of its assigned points. In this example, F_1 and F_2 (both low leverage, higher margin) are closest to the first center, while F_3 and F_4 (both high leverage, low margin) are closest to the second, giving updated centers $\mu_1 = (0.25, 0.275)$ and $\mu_2 = (0.85, 0.075)$. Recomputing distances with these centers changes no assignment, so the algorithm has converged after one update: the data separate into a "low-leverage, higher-margin" group and a "high-leverage, low-margin" group, purely from the geometry of the two features. Figure 6.10 shows the four firms, the initial centers, and the updated centers μ_1 and μ_2 together.

Wulin.Suo@Queensu.ca

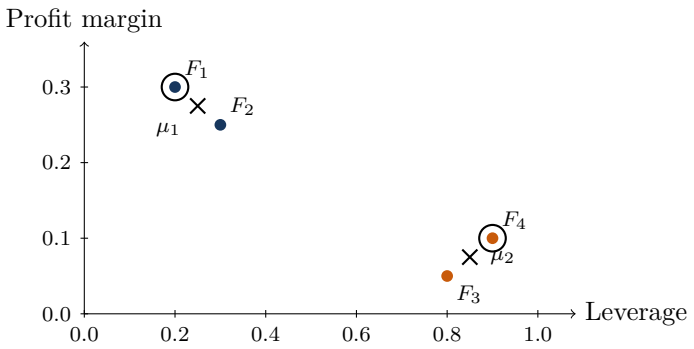


Figure 6.10: The K-means example: four firms in leverage/profit-margin space, with the initial centers (circled) and the updated centers μ_1 and μ_2

6.6.4 Neural Networks and Deep Learning in Finance

Suppose an analyst suspects that default risk depends on leverage and profitability through an interaction too intricate to write down in advance, and wants a model flexible enough to learn it from data rather than committing to a functional form as linear or logistic regression do. Neural networks are built for this kind of unspecified nonlinearity. A simple feedforward network with one hidden layer computes

$$\hat{y} = \sigma_2\big(W_2\,\sigma_1(W_1 x + b_1) + b_2\big),$$

where W_1, b_1, W_2, b_2 are learned weight matrices and bias vectors, and σ_1, σ_2 are nonlinear activation functions, such as the rectified linear unit $\mathrm{ReLU}(z) = \max(0, z)$ for hidden layers or, for a binary output layer, the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

which maps any real number into the interval $(0, 1)$ so that the output can be read as a probability, and which is the same function logistic regression applied in Section 6.3.4 above. The subscripted σ_1 and σ_2 stand for whichever activation a given layer happens to use; a bare $\sigma(\cdot)$, here and in every later chapter, means this sigmoid specifically. Stacking more such layers produces a deep network. The weights are learned by minimizing a loss (mean squared error for regression, log-loss for classification) with gradient descent, the gradients computed by backpropagation, which applies the chain rule layer by layer from the output back to the input.

A small numerical example makes the forward pass (computing a prediction from an input) concrete. Suppose a network has two inputs, a hidden layer with two ReLU units, and a single sigmoid output, with weights

$$W_1 = \begin{pmatrix} 0.4 & -0.2 \\ 0.1 & 0.3 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0.05 \\ -0.1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 0.6 & -0.5 \end{pmatrix}, \quad b_2 = 0.1.$$

For an input $x = (1.0, 0.5)$, the hidden layer's pre-activation values are $W_1 x + b_1 = (0.35, 0.15)$, and since both are already positive, the ReLU activation leaves them unchanged: $h = (0.35, 0.15)$. The output layer's pre-activation is $W_2 h + b_2 = 0.6(0.35) - 0.5(0.15) + 0.1 = 0.235$, and applying the sigmoid function gives the final prediction $\hat{y} = 1/(1 + e^{-0.235}) \approx 0.558$.

A modern deep network uses its millions of parameters the same way, matrix multiplications and elementwise nonlinearities repeated layer after layer; its power comes not from any single computation but from learning weights across many layers that jointly capture nonlinear structure. Figure 6.11 lays out this network, with every edge and bias labeled with its weight and every neuron with the value it computes.

6.6.5 Worked Example 6.10: Backpropagation, One Layer at a Time

Section 6.3.3 introduced gradient descent for a single-layer linear regression, where the gradient of the loss with respect to each parameter could be written down directly. A multi-layer network needs the chain rule

Wulin.Suo@Queensu.ca

to propagate a gradient backward through each layer in turn, and the toy network above makes every step concrete. Suppose this network is being trained as a classifier and the true label for input $x = (1.0, 0.5)$ is $y = 1$ (a positive case, using the log-loss $L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$ standard for classification). For this pairing of sigmoid output and log-loss, the gradient of the loss with respect to the output pre-activation z_2 collapses to $\hat{y} - y$:

$$\frac{\partial L}{\partial z_2} = \hat{y} - y = 0.5585 - 1 = -0.4415.$$

Backpropagation now works backward one layer at a time, applying the chain rule at each step. The output layer's weights and bias receive

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} h^\top = -0.4415\,(0.35,\ 0.15) = (-0.1545,\ -0.0662), \qquad \frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z_2} = -0.4415.$$

To continue backward into the hidden layer, the gradient must first pass through W_2 to reach h, then through the ReLU activation to reach the hidden layer's pre-activations z_1:

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} W_2 = -0.4415\,(0.6,\ -0.5) = (-0.2649,\ 0.2208), \qquad \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \mathrm{ReLU}'(z_1),$$

where $\mathrm{ReLU}'(z) = 1$ if $z > 0$ and 0 otherwise, and $\odot$ denotes elementwise multiplication; since both hidden pre-activations (0.35 and 0.15) are positive, $\mathrm{ReLU}'(z_1) = (1, 1)$ here and $\partial L / \partial z_1$ passes through unchanged. Finally, the first layer's weights and bias receive

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} x^\top = \begin{pmatrix} -0.2649 \\ 0.2208 \end{pmatrix} (1.0,\ 0.5) = \begin{pmatrix} -0.2649 & -0.1325 \\ 0.2208 & 0.1104 \end{pmatrix}, \qquad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} = (-0.2649,\ 0.2208).$$

All four gradients came from repeated application of the chain rule, moving backward from the loss through the output and hidden layers and reusing quantities already computed. That reuse is what makes backpropagation efficient enough for networks with millions of parameters; the alternative, perturbing each weight and re-running the forward pass, is computationally infeasible.

A gradient descent step with learning rate $\eta = 0.1$, the rule of Section 6.3.3, moves every parameter a small step opposite its gradient, $W_2 \leftarrow W_2 - \eta\, \partial L / \partial W_2$ and likewise for b_2, W_1, and b_1, nudging the prediction for this example toward $y = 1$. Repeating this update over many examples and iterations is how a deep network's weights are learned.

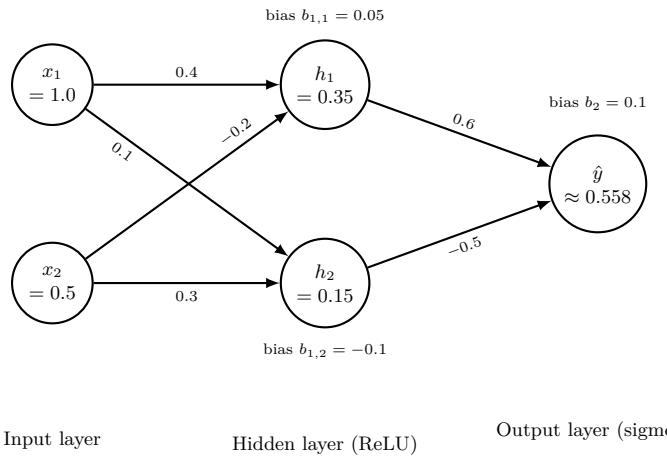


Figure 6.11: The toy feedforward network worked through above. Chapter 18 revisits it to verify LIME and integrated-gradients explanations against its exact analytical gradient.

In finance, neural networks have been used for forecasting, option pricing, time series modeling, and text analysis. Deep learning can learn nonlinear representations from raw data, reducing the need for hand-crafted

Wulin.Suo@Queensu.ca

features, but it is no magic solution. Networks often need substantial data, careful tuning of architecture and learning rate, and strong regularization (such as dropout or weight decay, which is equivalent to the ridge penalty) to avoid overfitting. They are also harder to interpret than a linear model or a single tree, which matters when the model influences real, sometimes regulated, decisions. Deep learning is therefore most useful with domain knowledge and robust validation; a network that does well in the laboratory can still fail in production if the data environment shifts or the model is not monitored.

6.7 Challenges of Machine Learning in Finance

Several challenges make financial applications of machine learning harder than many textbook demonstrations suggest.

Low signal-to-noise ratio. Financial returns are dominated by noise relative to any predictive signal, unlike many image or language tasks where the target is nearly deterministic given the input. An R^2 of a few percent can be economically meaningful for return prediction, whereas the same R^2 would be considered a failure in the clean regression example of Section 6.3.1.

Non-stationarity. The statistical relationships a model learns can change as markets evolve, as regulation changes, or as other participants adapt their own strategies. A model is trained on the past and asked to perform in a future that may not resemble it, a harder problem than the standard assumption that training and test data come from the same distribution.

Multiple testing and backtest overfitting. When researchers try many features, specifications, or trading rules and report only the best, the reported performance is biased upward, sometimes dramatically. This is the overfitting of Section 6.4.1 operating across a research process rather than a single fit, and one of the most persistent sources of disappointing live performance after a promising backtest.

Imbalanced outcomes. Events such as default, fraud, or market crashes are rare relative to normal outcomes. A classifier that always predicts "no event" can achieve high accuracy while being useless, which is why the precision, recall, and related metrics of Section 6.3.4 matter more than accuracy here. Remedies include resampling the training data (oversampling the rare class or undersampling the common one), assigning a higher misclassification cost to the rare class, and choosing the classification threshold from the relative costs of false positives and false negatives rather than defaulting to 0.5, which implicitly treats the two errors as equally costly.

Interpretability and regulatory scrutiny. Many financial applications, particularly credit decisions, are subject to regulation requiring that decisions be explainable to the affected individual and auditable by a regulator. This constrains the use of highly flexible models, such as a large ensemble or a deep neural network, or requires supplementing them with explanation methods, the subject of Chapter 18.

Feedback and reflexivity. Unlike physical systems, markets react to a successful model: if a profitable pattern becomes known and widely traded, the trading can erode or eliminate it. Past predictive power is therefore not a static property of the data; deploying the model can change it.

Hyperparameter and model-selection overfitting. Applied carelessly, Section 6.4.3's grid search is itself a source of overfitting: searching many hyperparameter combinations or model families and reporting only the best cross-validated result inflates performance like the multiple-testing problem above, one level removed.

Computational cost at scale. Some of the most powerful techniques in this chapter, an exhaustive grid search, a large gradient-boosted ensemble, a deep neural network, are expensive to train and retrain. That matters because financial models often need refitting daily or even intraday to keep up with a non-stationary market, an engineering constraint on top of the statistical ones.

These challenges do not make machine learning inapplicable to finance; they make careful validation, economically grounded evaluation, and ongoing monitoring non-negotiable parts of any deployment.

6.7.1 Case Vignette: Zillow Offers and the Winner's Curse

Zillow Group, the U.S. real-estate listings company whose Zestimate had popularized automated home valuation, began buying homes directly in 2018. Through Zillow Offers it made cash offers, bought the homes whose owners accepted, renovated them, and resold them, so the pricing model's offer was the business. On November 2, 2021, Zillow's board decided to wind Zillow Offers down. In the third quarter the company

Wulin.Suo@Queensu.ca

had bought 9,680 homes and sold 3,032; it recorded a $304.4 million write-down of inventory, having bought homes "at higher prices than Zillow Group's current estimates of future selling prices," expected a further $240 million to $265 million of charges on homes it was under contract to buy, and announced a reduction of about 25% of its workforce. Its shareholder letter described the problem in forecasting terms: the business depended on forecasting home prices "accurately three to six months into the future," and "we have been unable to accurately forecast future home prices" (Zillow Group 2021).

Two mechanisms from this chapter show how a valuation model that is accurate on average can still lose money systematically. The first is the **winner's curse**, a form of the selection bias in Equation (6.11): a seller knows things about the home that the model does not and tends to accept the offers that are too high. In the companion notebook's simulation, a price model whose errors average zero, with a standard deviation of 5%, makes offers on 100,000 homes, and each seller accepts only an offer above the home's true value. Across all the homes it priced, the model's mean error is +0.16%, essentially zero, so it would pass any validation on the homes it was asked to price. On the 50.4% of homes it actually buys, it overpays by 4.13% on average, close to the small-error approximation $\sigma\sqrt{2/\pi} = 3.99\%$, and the overpayment grows in proportion to the model's error: 1.62% for a 2% error and 8.54% for a 10% error. The second mechanism is non-stationarity: a model built while prices rose steadily was asked to forecast months ahead through a turn in the market, the regime shift that in Section 6.4.5 cut the coverage of a nominal 90% interval to 61%.

Neither mechanism requires a badly built model, and the public record does not say how much each contributed at Zillow; the case shows where validation has to look. A model whose output decides which transactions happen, a pricing, lending, or underwriting model, should be evaluated on the transactions it wins, not only on the ones it scores, and its offers should be shaded for the adverse selection they attract, the same reasoning a market maker applies when quoting to better-informed traders in Chapter 14. Chapter 9's reject-inference problem is the lending version of the same selection effect.

6.8 Key Takeaways

Machine learning offers valuable tools for prediction, classification, and pattern discovery in finance, but its power must be balanced with caution. The regression and classification examples showed how ordinary least squares, logistic regression, the confusion matrix, and R^2 are computed, so that later chapters can build on precise foundations. The fitted logistic regression showed what its coefficients and probabilities mean, the AUC turned out to be a count of correctly ordered pairs, and the Brier score, Platt scaling, and isotonic regression showed that a model can rank well and still need its probabilities repaired. Overfitting, unstable relationships, and shifting regimes threaten performance in practice, and the bias-variance decomposition explains why. The most successful applications combine statistical learning with domain knowledge, careful validation, and a clear understanding of the financial decision being made.

The chapter covered a range of approaches: parametric models fit by a closed form (linear regression) or by gradient descent (logistic regression and neural networks), nonparametric models that reason from stored examples (KNN), margin-based classifiers (SVM), probabilistic classifiers built on Bayes' theorem (naive Bayes), and ensembles of trees combined in parallel to reduce variance (random forests) or sequentially to reduce bias (gradient boosting). None is universally best. The choice depends on the size and structure of the data, the importance of interpretability (Chapter 18), and the computational constraints of deployment, which is why Section 6.4.3's cross-validated approach to model and hyperparameter selection matters as much as the algorithms themselves.

Two further distinctions shape how these models should be used. A point forecast is incomplete without its uncertainty: split conformal prediction (Section 6.4.5) wraps any model in intervals with a finite-sample, distribution-free coverage guarantee, which holds only while tomorrow's data are exchangeable with the calibration data, as regime shifts are not. And a model that predicts well has learned associations, not causal effects: in Section 6.4.6's credit-limit example a confounder made a harmful intervention look beneficial, a distortion that only a randomized experiment, or an observational design whose assumptions hold, can undo.

6.9 Suggested Exercises

6.1 Using the data in Table 6.1, verify by hand that $\hat{\beta}_1 = 47$ and $\hat{\beta}_0 = 20$, and compute the fitted spread for a firm with debt/EBITDA of 4.5.

Wulin.Suo@Queensu.ca

6.2 Add a sixth observation, debt/EBITDA of 7 and a credit spread of 500 bps, to Table 6.1, refit the regression, and discuss how much the slope and R^2 change. What does this reveal about the sensitivity of OLS to outliers?

6.3 Using Table 6.3, suppose the lender lowers its classification threshold so that applicant 5 is now also predicted to default. Recompute the confusion matrix and all five metrics, and discuss how precision and recall trade off against each other.

6.4 Explain the difference between supervised and unsupervised learning, giving one financial example of each that is not already used in the chapter.

6.5 Derive, in your own words, why a ridge penalty with a very large λ drives all coefficients toward zero, and explain what this implies about the bias and variance of the resulting model.

6.6 Describe the bias-variance trade-off using the formal decomposition in Section 6.4.2, and relate each of the three terms to a specific row of Table 6.6.

6.7 Why is walk-forward validation generally preferred over random k-fold cross-validation for financial time series?

6.8 Discuss one reason a model that performs well in-sample or in a backtest may fail out of sample or in live trading, drawing on at least two of the challenges described in Section 6.7 ("Challenges of Machine Learning in Finance").

6.9 Starting from $\beta_0 = \beta_1 = 0$, manually compute the first gradient descent update (Section 6.3.3) for the credit-spread data using a learning rate of $\eta = 0.02$ instead of 0.01, and compare the resulting (β_0, β_1) to the first iteration shown in Table 6.2.

6.10 Using Table 6.5, classify a new applicant with a debt-to-income ratio of 22% and 6 years of credit history using $k = 1$ and again using $k = 5$, and explain why the two predictions differ.

6.11 Explain, in your own words, why a wider margin in a support vector machine is generally associated with better generalization to new data, drawing an analogy to the bias-variance trade-off.

6.12 Choose one hyperparameter from any model in this chapter (for example, λ, k, or the number of boosting trees) and describe a grid of candidate values you would try, along with why searching too wide a grid could itself introduce overfitting risk, using Section 6.4.3 and Section 6.7's discussion of model-selection overfitting.

6.13 Explain why the naive independence assumption in naive Bayes is more clearly violated for the financial-ratio features used elsewhere in this chapter than for the word-count features used in text classification, and discuss whether you would still expect naive Bayes to be a reasonable baseline for a financial application.

6.14 Using this chapter's maximum-margin example, suppose a fifth point is added at $x = 3.5$ belonging to the "low-risk" class. Compute its functional margin under the original hyperplane ($w = 1$, $b = -3$), explain why this point must become a new support vector, and recompute the resulting hyperplane and margin width.

6.15 Using this chapter's ridge-versus-lasso worked example, explain in your own words why lasso's exact zero for the spurious feature's coefficient is a direct consequence of the penalty term's shape (a kink at zero), rather than simply a coincidence of this particular dataset, and describe what would need to be true of ridge's penalty for it to ever produce an exactly zero coefficient.

6.16 Using this chapter's backpropagation worked example, recompute $\partial L/\partial z_2$, $\partial L/\partial W_2$, and $\partial L/\partial b_2$ if the true label were $y = 0$ instead of $y = 1$ (all forward-pass values unchanged), and explain in words why the sign of every downstream gradient flips as a result.

Wulin.Suo@Queensu.ca

6.17 Using Table 6.8, construct the split conformal interval for the issuer at 4.5 turns of leverage at $\alpha = 0.20$ and at $\alpha = 0.05$, stating k, $\hat{q}$, and the exact coverage $k/(n+1)$ in each case. Then explain why a calibration set of 24 issuers cannot deliver a finite interval at $\alpha = 0.02$, and find the smallest calibration set that can.

6.18 The stress regime in Section 6.4.5 cut the coverage of a nominal 90% interval to 61.04%. Identify the assumption that failed, explain why recalibrating on the most recent observations helps once a new regime has persisted but cannot help on the first day of the shift, and describe how a risk manager could monitor realized coverage to detect the problem.

6.19 In Section 6.4.6's credit-limit example, suppose the bank's rule were more aggressive, $\Pr(D = 1 \mid Z) = 0.5 + 0.5Z$. Show that the population gap in average creditworthiness between treated and untreated customers becomes $2/3$, and use Equation (6.12) to compute the population value of the naive estimate. Under the original rule, how large would the true effect τ have to be for the naive estimate to be exactly zero?

6.20 A card issuer wants to test whether proactive credit-limit increases raise twelve-month default. Using the standard-error formula from Section 6.4.6's randomized estimate, and assuming default rates near 8.5% in both arms, estimate how many customers per arm are needed for a standard error of 0.0025. Discuss two practical or regulatory constraints on running the experiment, and explain why a predictive default model trained on the issuer's historical data, however accurate, cannot answer the question.

6.21 Using the fitted logistic regression of the worked example on Table 6.3, $\log(\hat{p}/(1-\hat{p})) = -7.2885 + 0.2063 \times \text{DTI}$, compute the default probability of applicants with debt-to-income ratios of 30% and 42%, and find the ratio at which the fitted probability equals 20%. If the lender declines every applicant above that ratio, which of the ten applicants are declined, and what are the resulting precision and recall?

6.22 Suppose applicant 9's debt-to-income ratio in Table 6.3 were 34% instead of 31%. Without refitting the model, compute the new AUC by counting concordant pairs, and explain why no refit is needed for this calculation but would be needed to recompute the confusion matrix at the 0.5 threshold.

6.23 Using the worked example on splitting a decision tree, verify the weighted Gini impurity of the splits at $t = 42$ and $t = 20$, and explain why the split at $t = 42$, which produces a perfectly pure node, still loses to the split at $t = 28.5$. Then suggest the best second split of the node containing applicants 1 to 6 and 9 (those above 28.5%).

6.24 Standardize the two features of Table 6.5 as in the feature-scaling worked example and classify the Exercise 10 applicant (debt-to-income 22%, 6 years of credit history) with $k = 1$ and $k = 5$. Compare your answers with those of Exercise 10 in raw units, and explain which feature dominated the raw distances.

6.25 A default model assigns 10% to 40 borrowers, of whom 3 default, and 50% to 10 borrowers, of whom 4 default. Compute its Brier score, then the Brier score after replacing each group's forecast by the group's observed default rate. Explain why a recalibration of this kind cannot raise the model's AUC.

6.26 A model predicts each stock's return over the next 20 trading days from daily features, and is evaluated with 5-fold cross-validation on contiguous blocks of 250 trading days. Explain which training observations must be purged around a validation block and why, how long an embargo you would set, and what fraction of the training data purging removes in each fold.

6.27 In the Zillow Offers vignette, the simulated overpayment on homes bought was 4.13% for a 5% model error. Using the small-error approximation $\sigma\sqrt{2/\pi}$, estimate the overpayment for a 10% error and compare it with the exact lognormal value of 8.51%. Explain why validating the model on all homes it priced would not reveal the problem, and propose a validation that would.

Wulin.Suo@Queensu.ca

6.28 **Challenge (real data).** Download the UCI Bank Marketing dataset (a Portuguese bank's telemarketing campaign, roughly 45,000 client contacts, with features such as age, job, account balance, and prior campaign outcomes, and a binary label for whether the client subscribed to a term deposit). (a) Fit both a logistic regression and a k-nearest-neighbors classifier (trying at least three values of k) to predict subscription on a held-out test split, and compare their accuracy, precision, and recall. (b) Using Section 6.4.2's bias-variance framework, explain which of your k values appears to underfit and which appears to overfit, based on how training and test accuracy diverge. (c) This dataset's subscription rate is only around 11%, far from the roughly balanced toy examples used elsewhere in this chapter; discuss, referencing this chapter's overfitting and validation material, why a model that reports 89% accuracy on this dataset could still be practically useless to the bank's marketing team.

6.10 Further Reading

- Hastie et al. (2009), *The Elements of Statistical Learning*. The standard advanced reference for the statistical theory behind essentially every model in this chapter, including a much more rigorous treatment of the bias-variance decomposition and regularization.
- Murphy (2012), *Machine Learning*. A thorough, probability-first treatment that develops naive Bayes, logistic regression, and neural networks within a unified Bayesian framework, extending the Bayes' theorem introduction from Chapter 1.
- James et al. (2023), *An Introduction to Statistical Learning*. A more accessible companion to Hastie et al. (2009), with the same authors and worked examples in R and Python; a good starting point before attempting the more mathematically demanding treatment.
- Hull et al. (2025), *Machine Learning in Business*. A business- and finance-oriented introduction to many of this chapter's models, by the same Hull whose derivatives and risk-management texts are cited in Chapters 1, 3, 5, 8, and 9, with less formal derivation and more emphasis on practical business use cases than the theory-first references above.
- López de Prado (2018), *Advances in Financial Machine Learning*. Focuses specifically on the financial-data pitfalls emphasized throughout this chapter, backtest overfitting, walk-forward validation, and feature construction for financial time series, in far greater depth than a single chapter allows.
- Cristianini and Shawe-Taylor (2000), *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods*. A focused treatment of the margin-maximization and kernel-trick ideas introduced only briefly in this chapter.
- Angelopoulos and Bates (2023), "Conformal Prediction: A Gentle Introduction." A short, practical introduction to conformal prediction, with code, covering the split method of Section 6.4.5, conformalized quantile regression for intervals that adapt to heteroskedasticity, and extensions to distribution shift.
- Angrist and Pischke (2009), *Mostly Harmless Econometrics: An Empiricist's Companion*. The standard applied guide to estimating causal effects from data, covering the regression adjustment, instrumental-variables, and difference-in-differences designs pointed to in Section 6.4.6.

Wulin.Suo@Queensu.ca